

THE HAVEN PROTOCOL

DOCUMENT 2

STATELESS TRANSLATION ADAPTER SPECIFICATIONS

STAS

Stateless Translation Adapter Specifications

Document Classification: Core Protocol Infrastructure

Architecture Domain: Clinical Interoperability / Edge Translation / Web3 Governance

Deployment Model: Client-Local Edge Infrastructure

Network: Polygon EVM

Primary Standard: HL7® FHIR®

Protocol State Model: Stateless / Event-Driven

Execution Model: Asynchronous / Fail-Closed

Version: STAS v2.0

Status: Production Architecture Specification

CONFIDENTIAL INSTITUTIONAL Technical Document. This architecture suite is published openly on GitHub and is licensed strictly under the terms of the GNU Affero General Public License (AGPL-3.0). Unauthorized derivative commercial execution or private re-licensing without an enterprise waiver from the author is strictly prohibited.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

DOCUMENT PURPOSE

This specification defines the technical architecture, deployment requirements, interface contracts, security boundaries, translation mechanisms, and operational controls governing the **Haven Stateless Translation Adapter (STA)**.

The STA is the protocol's client-side execution boundary between heterogeneous healthcare infrastructure and the Haven coordination rail.

It is designed to operate inside or adjacent to a participating organization's local infrastructure without requiring Haven to assume ownership of the organization's clinical database, EMR, laboratory system, pharmacy platform, HMO clearing environment, or internal application stack.

The adapter performs five fundamental functions:

1. **Legacy Data Ingestion**
Establishes controlled read/write interfaces with supported SQL and NoSQL healthcare systems.
2. **Schema Translation**
Converts heterogeneous clinical and administrative records into normalized FHIR-compatible representations.
3. **Cryptographic Event Packaging**
Converts normalized event state into cryptographically verifiable protocol payloads without transferring unnecessary source-system data.
4. **Protocol Transmission**
Submits authorized event blocks to the Haven coordination network through defined asynchronous interfaces.
5. **Economic Synchronization Enforcement**
Continuously verifies the required Polygon protocol synchronization state and places the local translation pipeline into a fail-closed condition when the required state is absent.

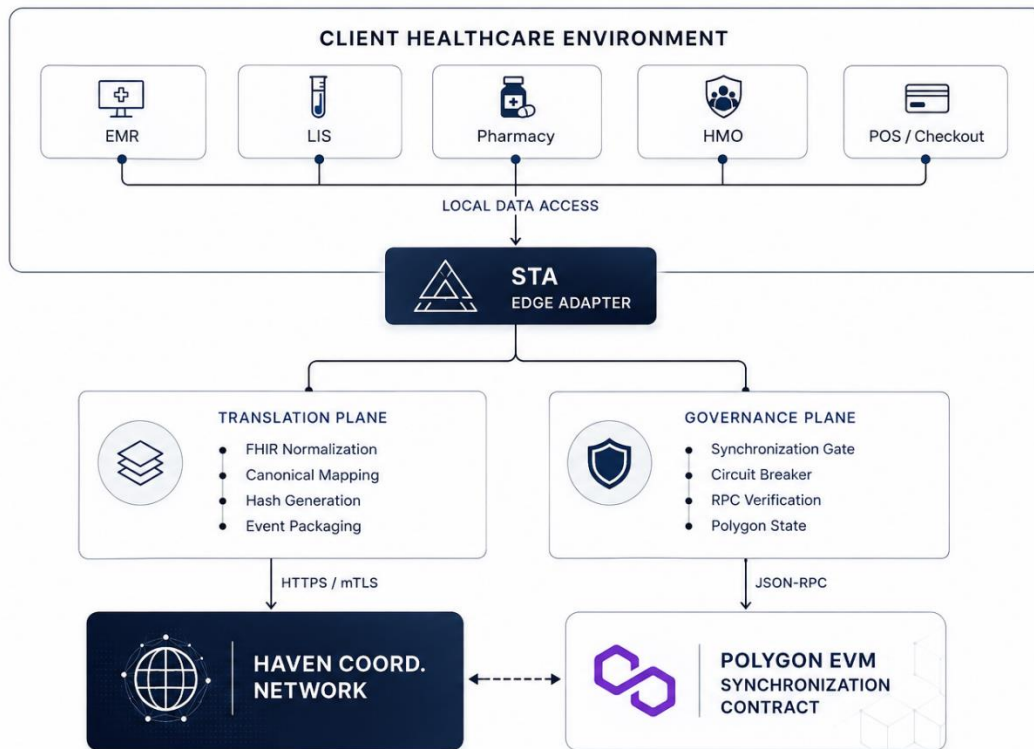
The STA therefore does **not** function as an EMR, central database, clinical application, payment processor, or custodial repository.

It is an **execution-bound translation and governance component**.

1.0 ARCHITECTURAL POSITION

The Stateless Translation Adapter shall be deployed at the **enterprise edge** of every participating healthcare environment.

The canonical deployment topology is:



1.1 Deployment Boundary

The STA shall remain logically separated from the client's primary application layer.

The client application must not be granted authority to modify:

- synchronization verification logic;
- circuit-breaker thresholds;
- contract addresses;
- contract ABI definitions;
- cryptographic verification rules;
- protocol routing destinations;
- event serialization rules;
- immutable protocol configuration;
- fee-routing verification parameters.

The adapter's configuration shall therefore be divided into:

A. Client Configuration

Permitted to contain:

- database connection metadata;
- local service endpoints;
- supported source-system identifiers;
- field mapping configuration;
- local logging preferences;
- operational retry parameters.

B. Protocol Configuration

Controlled by the Haven deployment package and cryptographically integrity-protected:

- Polygon chain identifier;
- synchronization contract address;
- expected contract interface;
- synchronization-state lookup method;
- circuit-breaker state machine;
- protocol endpoint;
- payload validation requirements;
- cryptographic domain parameters.

Client configuration must never be capable of overriding protocol configuration.

1.2 Fail-Closed Principle

The STA shall implement the following invariant:

No verified protocol synchronization state → No cross-network translation.

The circuit breaker therefore operates **before outbound protocol transmission**, not after the event has already crossed the interoperability boundary.

A synchronization failure shall not automatically imply deletion of locally generated clinical data.

Instead, the adapter shall:

1. preserve the source record;
2. preserve the local event identifier;
3. stop outbound translation/transmission;
4. place the event into a local pending state;
5. continue controlled health monitoring;
6. resume transmission only after synchronization is independently re-established.

1.3 Statelessness Requirement

The STA shall not become the authoritative clinical system of record.

Persistent clinical storage remains under the participating organization's existing infrastructure.

The adapter may maintain only the minimum operational state required for:

- transaction idempotency;
- event sequencing;
- retry management;
- cryptographic verification;
- circuit-breaker state;
- synchronization telemetry;
- dead-letter handling.

Any local operational persistence must be treated as **ephemeral protocol state**, not as a replacement clinical database.

1.4 Database Integration Principle

The adapter shall support heterogeneous database engines through isolated connectors rather than direct coupling of the translation engine to a single vendor schema.

Canonical connector classes shall include:

SQL

└─ PostgreSQL

└─ MySQL / MariaDB

└─ Microsoft SQL Server

└─ Oracle-compatible systems

NoSQL

- └─ MongoDB

- └─ Couchbase

- └─ Document-oriented stores

- └─ Key-value/event-oriented stores

The connector layer shall expose a normalized internal interface:

SourceConnector

- └─ discoverSchema()

- └─ readRecord()

- └─ readIncrementalEvents()

- └─ acknowledgeEvent()

- └─ healthCheck()

- └─ close()

The translation engine must interact with this abstraction rather than embedding database-specific logic inside FHIR transformation routines.

This separation ensures that **database replacement does not require reconstruction of the protocol translation layer.**

EDGE DEPLOYMENT ARCHITECTURE

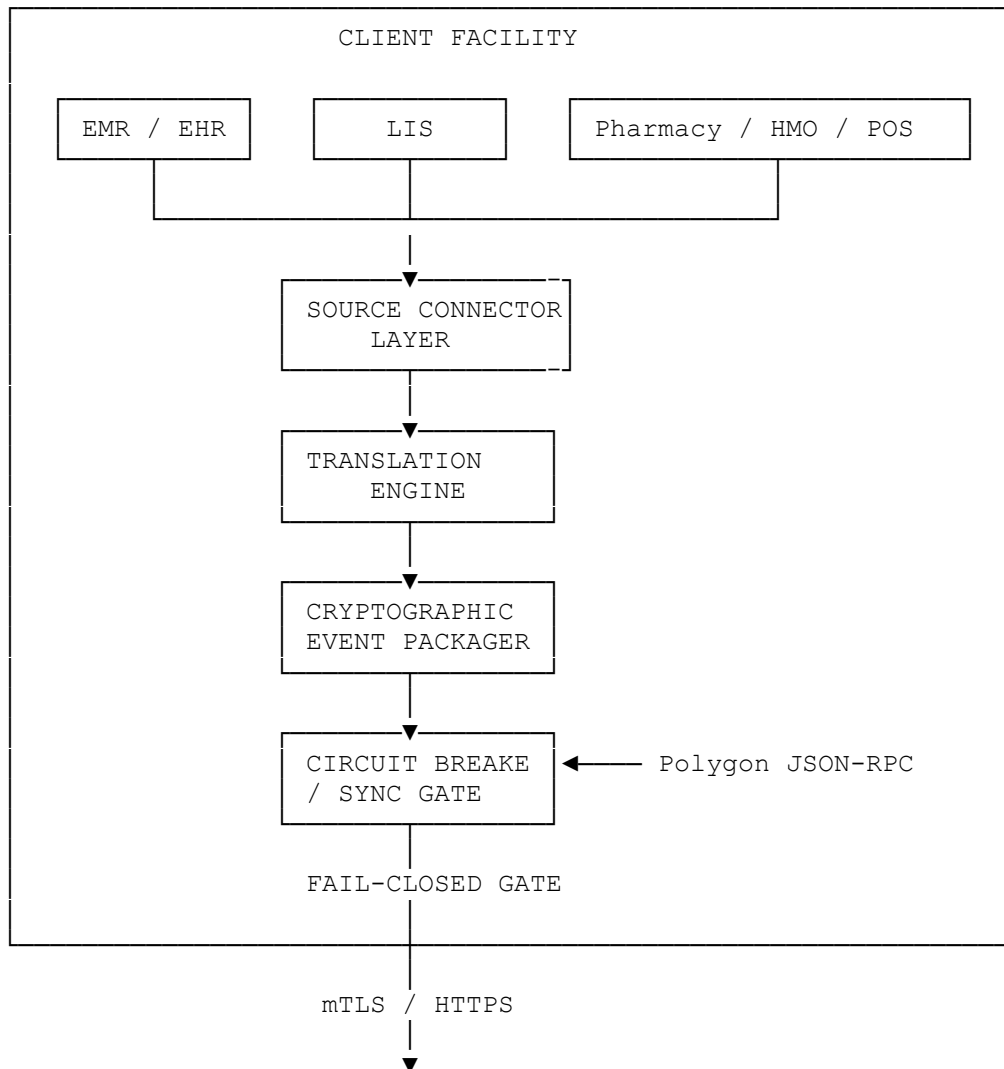
2.1 Deployment Model

The Haven Stateless Translation Adapter (STA) shall be deployed as an isolated edge service within the participating organization's controlled infrastructure.

The preferred deployment topology is a dedicated virtual machine, container host, or hardened bare-metal service positioned between the organization's internal healthcare systems and the Haven protocol boundary.

The STA SHALL NOT require modification of the source EMR application's core business logic.

The minimum logical deployment shall contain the following components:



2.2 Network Segmentation

The STA deployment SHALL use separate logical network zones.

Zone A — Source Systems

Contains:

- EMR database servers;
- laboratory information systems;
- pharmacy systems;
- HMO systems;
- legacy clinical applications;
- point-of-sale systems.

These systems SHALL NOT receive inbound connections originating from the public internet.

Zone B — Adapter Network

Contains:

- STA runtime;
- source connectors;
- translation engine;
- cryptographic event processor;
- local queue;
- circuit-breaker controller;
- protocol configuration.

This zone provides the controlled interoperability boundary.

Zone C — External Protocol Boundary

Provides controlled outbound connectivity to:

- Haven protocol endpoints;
- Polygon JSON-RPC endpoints;
- approved observability endpoints where required.

Inbound internet-originated connections to the STA SHALL be prohibited unless explicitly required by a documented operational function.

2.3 Database Credential Isolation

Database credentials SHALL be maintained independently for every source connector.

The STA SHALL NOT use:

- shared application administrator credentials;
- database superuser credentials;
- unrestricted root accounts;
- credentials capable of modifying unrelated schemas.

The preferred database permission model is:

```
STA_DATABASE_USER
├── SELECT permitted source tables
├── EXECUTE permitted stored procedures
├── INSERT acknowledgement records where required
└── NO schema administration
```

Where the source architecture supports change-data-capture or append-only event tables, the STA SHOULD consume those mechanisms rather than repeatedly scanning complete clinical tables.

2.4 Local Queue

The adapter SHALL maintain a bounded local event queue for asynchronous processing.

Each queued event SHALL contain at minimum:

```
event_id
source_system_id
source_record_id
event_type
event_timestamp
schema_version
translation_status
cryptographic_status
synchronization_status
retry_count
created_at
updated_at
```

Clinical source payloads SHALL NOT be duplicated unnecessarily within the queue.

Where possible, the queue SHALL contain a reference to the source record and a deterministic cryptographic representation of the required event state.

2.4.1 Bounded Queue Scale and Backpressure Controls

The local event queue SHALL operate within strict resource boundaries to prevent disk-space exhaustion or memory exhaustion attacks on host infrastructure during a sustained CIRCUIT_OPEN state.

- **Maximum Bounded Capacity:** The local queue storage ceiling SHALL be capped at exactly 50,000 serialized event records or 10 GB of total allocated disk storage, whichever limit is reached first.
- **Threshold Warning:** Upon reaching 80% of maximum bounded capacity (40,000 records or 8 GB), the adapter SHALL emit a high-priority operational telemetry log event: `'QUEUE_PRESSURE_WARNING'`.
- **Overcapacity Drop Policy:** If the queue reaches 100% capacity while the local circuit breaker remains OPEN, the adapter SHALL activate an explicit Fail-Closed Rejection policy. The translation engine SHALL reject new incoming clinical updates from the source connector, freeze upstream extraction cursors, and instantly return error code `'RESOURCE_EXHAUSTED'` to the client facility's execution runtime.
- **Memory Deflation:** The local adapter memory pool SHALL explicitly forbid drop-packet shedding or First-In, First-Out (FIFO) record eviction. Causal historical event ordering MUST remain intact until synchronization is independently re-established and the queue is safely drained via bounded retry backoff mechanisms.

2.5 Resource Constraints

The STA SHOULD be capable of operating on modest client infrastructure.

Target baseline:

CPU:	2 vCPU minimum
Memory:	2-4 GB minimum
Storage:	20 GB operational allocation
Network:	Stable outbound HTTPS connectivity
Runtime:	Linux-based hardened environment
Clock:	NTP-synchronized

Production sizing SHALL be determined by event throughput, database latency, concurrent connector count, and local queue depth.

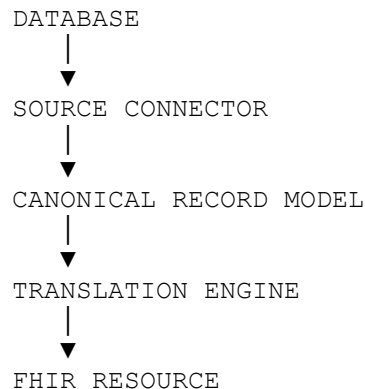
SQL / NOSQL SOURCE CONNECTOR SPECIFICATION

3.1 Connector Architecture

The STA SHALL isolate source-system access behind a connector abstraction.

No FHIR transformation function may directly execute database-specific queries.

The execution path SHALL be:



This prevents database-specific implementation details from propagating into the protocol layer.

3.2 SQL Integration

SQL connectors SHALL support parameterized queries exclusively.

The connector SHALL reject dynamically concatenated SQL statements derived from external or clinical input.

Canonical query execution:

```
prepare(statement)
bind(parameters)
execute()
validate(result)
normalize(result)
```

Example interface:

```
interface SqlConnector {
    healthCheck(): ConnectorHealth
    execute(query: ParameterizedQuery): QueryResult
    stream(query: ParameterizedQuery): EventStream
    acknowledge(eventId: String): AckResult
    close(): void
}
```

The connector SHALL support:

- PostgreSQL;
- MySQL/MariaDB;
- Microsoft SQL Server;
- Oracle-compatible relational systems.

Database-specific drivers SHALL remain isolated inside the connector implementation.

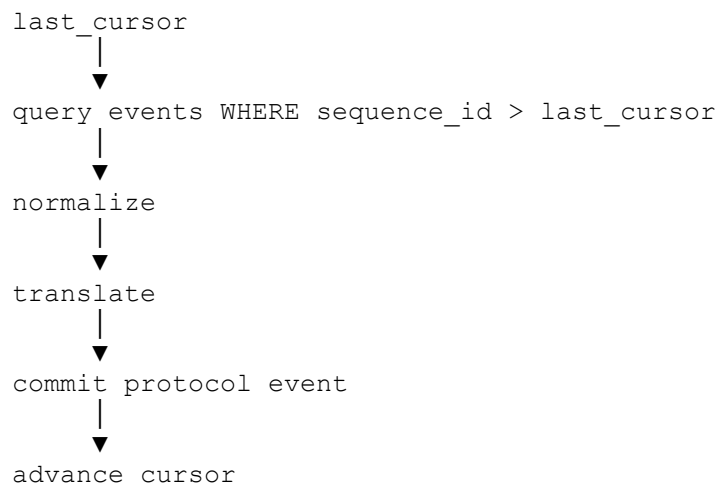
3.3 Incremental Extraction

The STA SHOULD prefer incremental extraction over full-table polling.

Supported mechanisms MAY include:

1. Timestamp cursor
2. Monotonically increasing sequence ID
3. Database change-data-capture
4. Transaction log consumer
5. Append-only event table
6. Source-system webhook

The preferred event cursor model is:



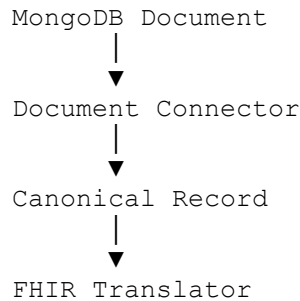
The cursor SHALL only advance after successful local processing.

This prevents an adapter restart from silently skipping source-system events.

3.4 NoSQL Integration

NoSQL connectors SHALL normalize document-oriented records into the same canonical internal representation used by SQL connectors.

Example:



The adapter SHALL support deterministic extraction of nested fields without assuming a rigid relational schema.

Where document structures vary between records, the connector SHALL preserve the original field path during normalization.

Example:

```
patient.contact[0].telecom[0].value
```

shall remain traceable to its source-system field path.

3.5 Canonical Record Envelope

Every source record entering the translation engine SHALL first be converted into a canonical envelope.

```
{
  "sourceSystem": "CLIENT_EMR",
  "sourceRecordId": "opaque-record-id",
  "eventType": "clinical.update",
  "eventTimestamp": "2026-08-30T00:00:00Z",
  "schemaVersion": "1.0",
  "fields": {},
  "provenance": {
    "connector": "postgresql",
    "sourceTable": "clinical_events",
    "sourceVersion": "v4"
  }
}
```

The `sourceRecordId` SHALL remain an opaque identifier.

The adapter SHALL NOT expose internal database primary keys to external parties unless explicitly required by an approved interoperability contract.

3.6 Connector Security Requirements

Each connector SHALL enforce:

- TLS where supported
- Credential isolation
- Least-privilege access
- Connection timeout
- Query timeout
- Maximum result size
- Connection pooling limits
- Retry backoff
- Circuit isolation
- Audit logging

A source database failure SHALL NOT cause uncontrolled retry amplification.

The adapter SHALL use bounded exponential backoff for recoverable connector failures.

Example:

- Attempt 1 → immediate retry
- Attempt 2 → 1 second
- Attempt 3 → 2 seconds
- Attempt 4 → 4 seconds
- Attempt 5 → 8 seconds
- ...
- Maximum backoff → configured operational ceiling

After the retry ceiling is reached, the event SHALL enter a controlled local failure queue.

No clinical event shall be discarded solely because the source database is temporarily unavailable.

FHIR-NATIVE TRANSLATION ENGINE

4.1 Translation Objective

The Translation Engine converts heterogeneous source-system records into standards-compliant HL7 FHIR resources without requiring the originating system to natively implement FHIR.

The engine SHALL operate deterministically.

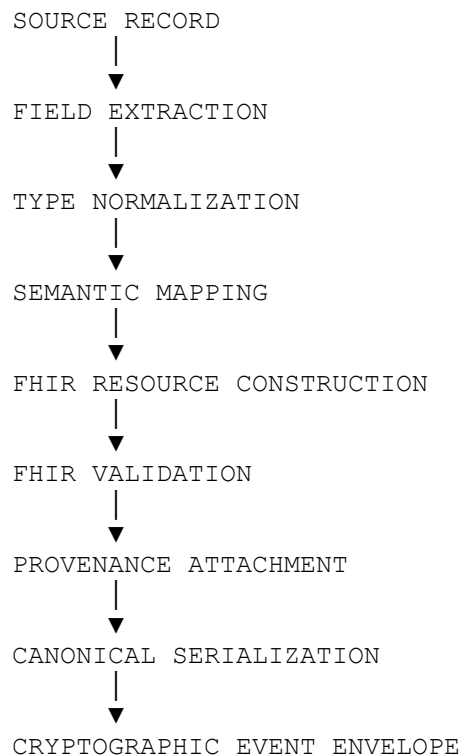
Identical source state, mapping configuration, and protocol version SHALL produce an equivalent canonical representation.

The engine SHALL NOT infer clinical facts that are not explicitly present in the source record.

Where a source value cannot be mapped with sufficient confidence, the field SHALL be marked unresolved rather than fabricated.

4.2 Translation Pipeline

The mandatory translation sequence is:



Each stage SHALL produce a machine-verifiable result.

Failure at any mandatory stage SHALL prevent the event from entering the outbound protocol queue.

4.3 Mapping Registry

FHIR mappings SHALL be maintained in a version-controlled mapping registry.

Example:

```
{
  "mappingVersion": "1.0.0",
  "sourceSystem": "CLIENT_EMR",
  "resource": "Patient",
  "rules": [
    {
      "source": "patient.id",
      "target": "identifier.value",
      "transform": "opaqueIdentifier"
    },
    {
      "source": "patient.date_of_birth",
      "target": "birthDate",
      "transform": "isoDate"
    },
    {
      "source": "patient.sex",
      "target": "gender",
      "transform": "administrativeGender"
    }
  ]
}
```

The mapping registry SHALL itself be integrity-protected.

A client application SHALL NOT be able to silently alter protocol-critical mappings.

Mapping changes SHALL increment the mapping version.

4.4 Core FHIR Resource Mapping

The minimum supported resource families SHALL include:

```
Patient
Encounter
Observation
Condition
Procedure
MedicationRequest
Medication
DiagnosticReport
ServiceRequest
Practitioner
```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Organization
Location
Coverage
Claim
Appointment

Additional resources MAY be implemented through independently versioned mapping modules.

4.5 Patient Identity Mapping

The STA SHALL NOT assume that a source-system patient identifier is globally unique.

Source identifiers SHALL therefore be namespaced.

Canonical identity construction:

```
namespace = source_system_id
identifier = source_record_id

canonical_reference =
    namespace + ":" + identifier
```

Example:

```
CLIENT_HOSPITAL_A:847291
```

The canonical identifier SHALL be treated as an opaque protocol identifier.

Where Haven's deployment architecture requires pseudonymous identity, the adapter SHALL derive the protocol identifier through a deterministic keyed cryptographic function rather than exposing the originating database identifier.

Conceptually:

```
protocol_id = HMAC-SHA256(
    tenant_scoped_secret,
    source_system_id || ":" || source_record_id
)
```

The secret SHALL remain within the authorized trust boundary.

4.6 Clinical Value Normalization

Normalization SHALL occur before FHIR validation.

Examples include:

Date:

DD/MM/YYYY → YYYY-MM-DD

Timestamp:

Local timestamp + validated timezone

→ RFC 3339 representation

Boolean:

YES / Y / TRUE / 1

→ true

Boolean:

NO / N / FALSE / 0

→ false

Unit normalization SHALL use an approved terminology and unit mapping registry.

Example:

mg/dL



Canonical UCUM representation

The engine SHALL preserve the original value and unit where required for provenance and auditability.

4.7 Unstructured Input Handling

Free-text clinical fields SHALL NOT be automatically converted into structured clinical assertions solely through uncontrolled language-model inference.

Where deterministic extraction is possible, the extracted value SHALL retain:

sourceField
sourceValue
extractionMethod
mappingVersion
confidence/status

Any clinically consequential automated interpretation SHALL require an explicitly authorized processing policy.

The default behavior for ambiguous unstructured content SHALL be:

UNRESOLVED

rather than an invented FHIR assertion.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

FHIR VALIDATION AND CRYPTOGRAPHIC EVENT CONSTRUCTION

5.1 FHIR Validation Gate

Every generated FHIR resource SHALL pass structural validation before cryptographic packaging.

Validation SHALL verify:

Resource type
FHIR version
Required fields
Datatype validity
Reference structure
Cardinality
Coding structure
Identifier structure
Timestamp format
Profile constraints

Invalid resources SHALL NOT be transmitted.

The adapter SHALL return a deterministic validation result:

```
{
  "valid": false,
  "resourceType": "Observation",
  "errors": [
    {
      "path": "Observation.status",
      "code": "required",
      "message": "Required field missing"
    }
  ]
}
```

5.2 Canonical Serialization

Before hashing or signing, the event SHALL be converted into a deterministic serialization.

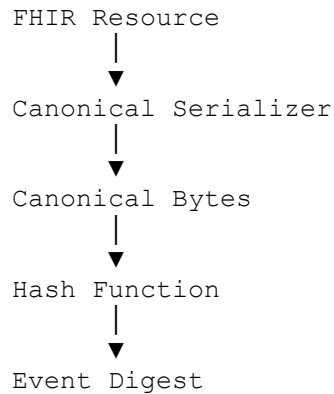
The serialization process SHALL eliminate non-semantic variance such as:

- inconsistent property ordering;
- unnecessary whitespace;
- unstable formatting;
- implementation-specific serialization artifacts.

The canonical serialized representation SHALL become the cryptographic input.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Conceptually:



5.3 Event Digest

The STA SHALL calculate a cryptographic digest for every protocol event.

Preferred construction:

```
event_digest =  
SHA-256(  
    protocol_version ||  
    source_system_id ||  
    event_type ||  
    event_timestamp ||  
    canonical_fhir_payload  
)
```

The exact concatenation format SHALL be specified by the protocol serialization standard and SHALL remain versioned.

The digest provides integrity verification.

It does not itself constitute proof of clinical truth.

5.4 Event Envelope

The canonical protocol envelope SHALL contain:

```
{  
  "protocolVersion": "1.0",  
  "eventId": "uuid",  
  "sourceSystemId": "CLIENT_EMR",  
  "eventType": "Observation.created",  
  "eventTimestamp": "2026-08-30T00:00:00Z",  
  "mappingVersion": "1.0.0",  
  "fhirVersion": "R4",  
  "resourceType": "Observation",  
}
```

```

    "payloadDigest": "sha256:...",
    "payload": {},
    "provenance": {
      "adapterVersion": "1.0.0",
      "connectorVersion": "1.0.0"
    }
  }
}

```

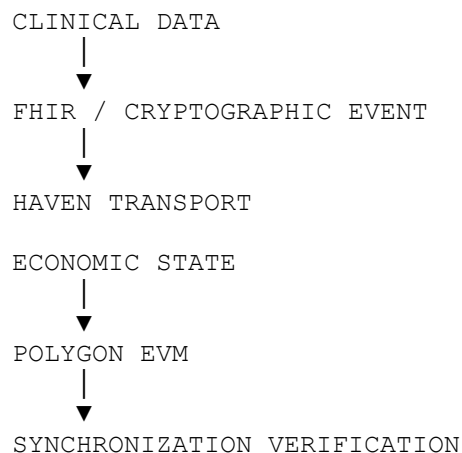
5.5 Separation of Clinical Payload and Settlement State

Clinical payload integrity and economic synchronization SHALL remain logically distinct.

The adapter SHALL NOT place private clinical data onto the Polygon ledger merely to prove that a protocol fee was settled.

The Polygon layer SHALL function as a **governance and synchronization verification mechanism**, while clinical payload transmission occurs through the designated interoperability transport.

Accordingly:



The two planes are cryptographically linked through protocol state and event identifiers where required, but sensitive clinical payloads SHALL remain outside the public blockchain.

5.6 Synchronization Gate Contract Interface

The STA SHALL maintain a protocol-defined contract address for synchronization verification.

The contract address SHALL be immutable within the protected protocol configuration.

The adapter SHALL query the required mapping state using a read-only JSON-RPC call.

Conceptual Solidity interface:

```
interface IHavenSynchronization {  
    function isNetworkSynchronized(  
        address participant  
    ) external view returns (bool);  
}
```

The adapter SHALL NOT accept a client-provided Boolean such as:

```
isNetworkSynchronized = true
```

as evidence of synchronization.

The authoritative value SHALL originate from the deployed Polygon contract.

5.7 Contract Identity Verification

Before accepting synchronization responses, the adapter SHALL verify:

```
Configured Chain ID  
Configured Contract Address  
Expected Contract Interface  
Expected Function Selector  
Expected Response Type
```

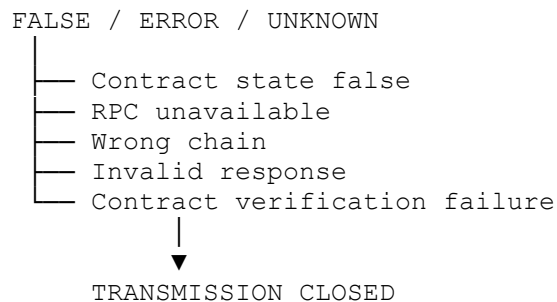
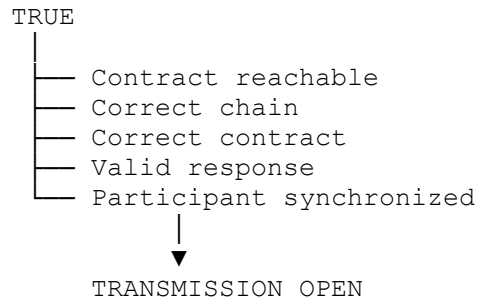
The adapter SHALL reject synchronization responses originating from an unexpected network or contract address.

A JSON-RPC response from an arbitrary contract SHALL never be treated as equivalent to the Haven synchronization state.

5.8 Synchronization State Semantics

The adapter SHALL recognize only explicitly valid protocol states.

Conceptually:



UNKNOWN SHALL NEVER BE INTERPRETED AS TRUE.

5.9 RECEIPT PROVENANCE MARK & CONSUMER INTEGRITY INVARIANT

Upon the successful cryptographic validation and asynchronous acknowledgment of any clinical state change (including `PATIENT_IDENTITY_ASSERTION`, `LAB_RESULT`, or `DISPENSATION`), the Stateless Translation Adapter shall return an authoritative provenance block to the client facility's execution runtime.

5.9.1 CANONICAL FOOTER STRING

The client's local record-rendering engine, receipt generation module, and print-out document formatters SHALL natively append the following exact, uppercase ASCII string parameter to the bottom margin of the issued end-user consumer artifact:

[COORDINATED BY HAVEN // SECURE INTEROPERABILITY UTILITY PROTOCOL
v1.0]

5.9.2 PROVENANCE ENFORCEMENT MOAT

The local Policy Enforcement Point (PEP) SHALL treat the presence of this provenance mark as a core metadata requirement. If the client facility's frontend or backend template scripts attempt to strip, alter, obfuscate, or omit the "Coordinated by Haven" parameter from the consumer output tier, the adapter shall return error code ``MAP_UNRESOLVED``, instantly toggle the local

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

circuit breaker to OPEN, and freeze all downstream cross-network data translations natively within 60 seconds.

No valid consumer print-out shall be generated without verified protocol provenance.

POLYGON SYNCHRONIZATION VERIFICATION ENGINE

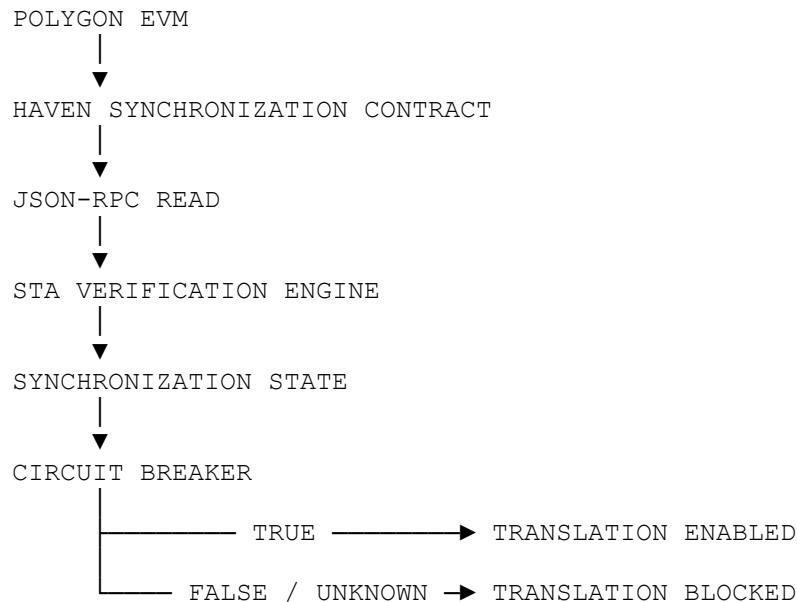
6.1 Purpose

The Synchronization Verification Engine establishes the adapter's independent determination of whether the participating network is authorized to continue cross-network translation.

The engine SHALL obtain synchronization state directly from the designated Polygon smart contract through a read-only JSON-RPC request.

No client application variable, browser response, checkout response, HTTP parameter, locally supplied Boolean, or client-side JavaScript value SHALL constitute synchronization evidence.

The authoritative verification chain is:



6.2 Polling Interval

The adapter SHALL execute a synchronization verification cycle at a nominal interval of **60 seconds**.

The 60-second interval SHALL be treated as a maximum intended detection interval under normal operating conditions, not as a guarantee of instantaneous network-wide enforcement.

A local circuit breaker MAY transition to OPEN immediately when an outbound operation receives an explicit synchronization failure.

The adapter SHALL NOT wait for the next scheduled poll when an explicit invalid state has already been established.

6.3 Unresolved Mapping and Fail-Closed Enforcement

The STA SHALL NOT generate a protocol-authorized cryptographic payload from a FHIR bundle containing unresolved structural mappings.

Any source field that cannot be deterministically mapped to an approved FHIR element SHALL produce an explicit unresolved-state condition.

The unresolved state SHALL be machine-readable and SHALL prevent the event from entering the outbound Haven transmission queue.

Reference unresolved-state schema:

```
UNRESOLVED_STATE_SCHEMA = {
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "status": {
      "type": "string",
      "enum": ["FAIL_CLOSED"]
    },
    "unmapped_fields": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "resource_type": {
      "type": "string"
    }
  },
  "required": [
    "status",
    "unmapped_fields",
    "resource_type"
  ]
}
```

The mapping engine SHALL perform recursive validation across all resources contained within the FHIR bundle.

At minimum, the validator SHALL inspect:

```
Encounter
Observation
Condition
Patient
Procedure
MedicationRequest
DiagnosticReport
Coverage
Claim
```

and any additional resource types supported by the active mapping profile.

Reference implementation:

```
from jsonschema import validate, ValidationError

def validate_and_hash_payload(fhir_bundle: dict) -> str:
    unresolved_fields = []

    for entry in fhir_bundle.get("entry", []):
        resource = entry.get("resource", {})
        resource_type = resource.get("resourceType")

        if not resource_type:
            unresolved_fields.append("resourceType")
            continue

        if "unmapped_keys" in resource:
            unresolved_fields.extend(
                resource["unmapped_keys"]
            )

    if unresolved_fields:
        unresolved_state = {
            "status": "FAIL_CLOSED",
            "unmapped_fields": unresolved_fields,
            "resource_type": resource_type or "UNKNOWN"
        }

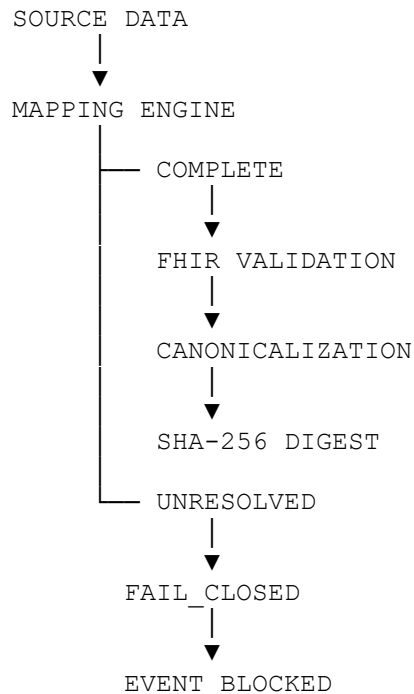
        validate(
            instance=unresolved_state,
            schema=UNRESOLVED_STATE_SCHEMA
        )

        raise ValidationError(
            "Unresolved structural mapping detected"
        )

    return calculate_sha256_digest(
        canonicalize_fhir_bundle(fhir_bundle)
    )
```

6.3.1 Fail-Closed Processing Rule

The mapping engine SHALL enforce:



An unresolved mapping SHALL therefore terminate the normal cryptographic packaging path.

6.3.2 No Digest Authorization

A digest SHALL NOT be interpreted as proof that an unresolved payload is valid.

The STA SHALL distinguish:

PAYLOAD INTEGRITY
≠
PAYLOAD SEMANTIC VALIDITY

A SHA-256 digest establishes integrity of a specific byte representation. It does not establish that the underlying FHIR representation is clinically or structurally valid.

Therefore:

INVALID FHIR
+
VALID SHA-256
=
INVALID PROTOCOL EVENT

6.3.3 Unmapped Field Registry

Every unresolved field SHALL be recorded using a deterministic field path.

Example:

```
Observation.component[0].legacy_lab_flag
Encounter.legacy_visit_category
Patient.old_hmo_reference
```

The adapter SHALL avoid logging complete clinical values when recording unresolved fields.

The preferred record is:

```
{
  "status": "FAIL_CLOSED",
  "unmapped_fields": [
    "Observation.component[0].legacy_lab_flag"
  ],
  "resource_type": "Observation"
}
```

6.3.4 Resource-Level Isolation

Where the protocol permits independent processing of resources within a bundle, an unresolved resource SHALL be isolated from successfully validated resources.

Example:

```
FHIR BUNDLE
├── Patient      → VALID
├── Encounter    → VALID
├── Observation  → UNRESOLVED
└── Condition   → VALID
```

The adapter SHALL NOT silently discard the unresolved Observation and transmit the remainder as though the original event were complete.

The active protocol transaction model SHALL determine whether the complete bundle is blocked or whether valid resources may be independently committed.

6.3.5 Error Classification

Unresolved mapping SHALL produce a deterministic error classification:

```
MAP_UNRESOLVED
```

The corresponding operational state SHALL be:

TRANSMISSION = BLOCKED
RETRY = NOT AUTOMATICALLY AUTHORIZED
REMEDICATION = REQUIRED

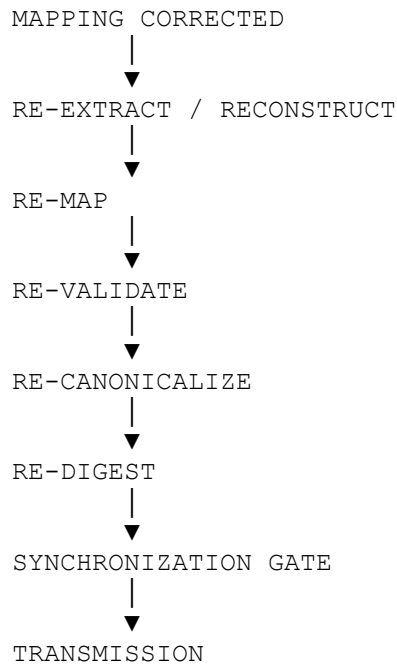
The event SHALL remain available for remediation without modifying the original source record.

6.3.6 Retry After Mapping Correction

A corrected mapping SHALL cause the event to re-enter the complete validation pipeline.

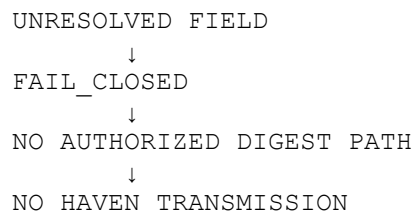
The adapter SHALL NOT simply reuse a previously generated digest.

Required sequence:



6.3.7 Mandatory Invariant

The STA SHALL enforce the following invariant:



This prevents incomplete legacy-system mappings from being converted into apparently valid protocol events merely because the resulting payload can be cryptographically hashed.

6.3.8 Implementation Requirement

The production implementation SHALL treat the unresolved state as a **control-plane security condition**, not merely as a data-quality warning.

Accordingly:

```
WARNING
    ≠
FAIL_CLOSED

LOGGED_ERROR
    ≠
BLOCKED_EVENT

HASHED_PAYLOAD
    ≠
AUTHORIZED_PAYLOAD
```

The STA SHALL only permit an event to proceed when structural mapping, FHIR validation, canonicalization, cryptographic processing, and protocol synchronization have all independently satisfied their required controls.

6.4 Contract Function Encoding

For a Solidity interface:

```
function isNetworkSynchronized(
    address participant
) external view returns (bool);
```

the adapter SHALL construct the appropriate ABI-encoded calldata.

Conceptual implementation:

```
function_selector =
keccak256("isNetworkSynchronized(address)")
    [first 4 bytes]

calldata =
function_selector ||
ABI.encode(participant_address)
```

The exact ABI encoding SHALL be generated using a standards-compliant EVM ABI library rather than manually implemented where a vetted library is available.

6.5 RPC Request Structure

Conceptual JSON-RPC request:

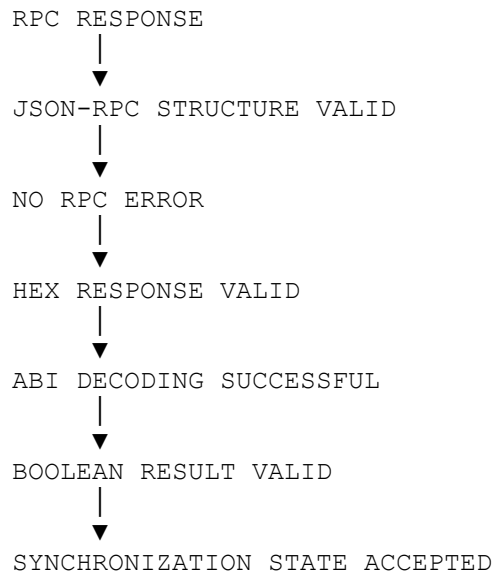
```
{
  "jsonrpc": "2.0",
  "id": 1001,
  "method": "eth_call",
  "params": [
    {
      "to": "0x<IMMUTABLE_SYNC_CONTRACT>",
      "data": "0x<ABI_ENCODED_CALLDATA>"
    },
    "latest"
  ]
}
```

The adapter SHALL validate the complete response.

A syntactically valid JSON-RPC response SHALL not automatically constitute a valid synchronization result.

6.6 Response Validation

The response validation sequence SHALL be:



Any failure in this chain SHALL result in an **UNKNOWN** synchronization state.

UNKNOWN SHALL be handled identically to an authorization failure for purposes of cross-network transmission.

6.7 Chain Verification

Before synchronization state is accepted, the adapter SHALL verify the connected chain.

The adapter SHALL periodically or during startup execute:

```
eth_chainId
```

and compare the returned chain identifier against the protocol's configured Polygon chain identifier.

A chain mismatch SHALL produce:

```
CHAIN_MISMATCH
```

and SHALL immediately prevent synchronization authorization.

The adapter SHALL NOT silently redirect itself to another network.

6.8 Contract Address Pinning

The synchronization contract address SHALL be pinned in protected protocol configuration.

Example:

```
polygon:
  chainId: "<EXPECTED_CHAIN_ID>"
  synchronizationContract:
    address: "0x<IMMUTABLE_CONTRACT_ADDRESS>"
    function: "isNetworkSynchronized(address) "
```

The application-facing configuration layer SHALL NOT expose a runtime mechanism for replacing this address.

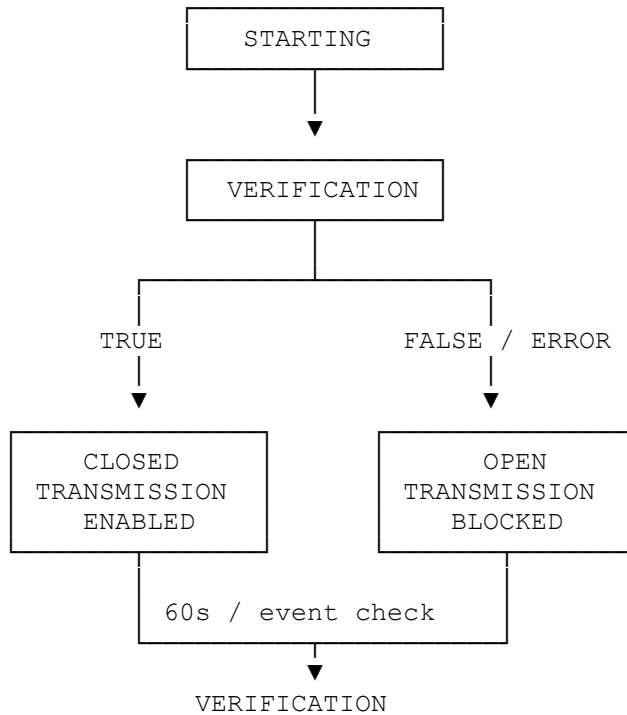
A contract-address modification SHALL require an authenticated protocol deployment procedure and a corresponding configuration version transition.

60-SECOND CIRCUIT BREAKER ENGINE

7.1 State Machine

The circuit breaker SHALL operate as a deterministic state machine.

Canonical states:



The states SHALL be interpreted as:

STARTING

No synchronization determination has yet been established.

Cross-network translation SHALL remain blocked.

CLOSED

A valid synchronization response has been independently verified.

Cross-network translation MAY proceed.

OPEN

Synchronization is false, unavailable, invalid, or unverifiable.

Cross-network translation SHALL be blocked.

7.2 Core Invariant

The adapter SHALL enforce:

```
ALLOW_TRANSLATION =  
    chain_verified  
    AND contract_verified  
    AND rpc_verified  
    AND synchronization_state == TRUE
```

No individual client-side variable SHALL be capable of overriding this expression.

Conceptually:

```
if (  
    chainVerified &&  
    contractVerified &&  
    rpcVerified &&  
    synchronized === true  
) {  
    circuitBreaker.close();  
} else {  
    circuitBreaker.open();  
}
```

7.3 Polling Worker

Reference implementation:

```
async function synchronizationLoop() {  
    while (adapterRunning) {  
        try {  
            const chain = await verifyChainId();  
  
            if (!chain.valid) {  
                circuitBreaker.open("CHAIN_MISMATCH");  
                await sleep(60000);  
                continue;  
            }  
  
            const contract = await verifyContractConfiguration();  
  
            if (!contract.valid) {  
                circuitBreaker.open("CONTRACT_INVALID");  
                await sleep(60000);  
                continue;  
            }  
        }  
    }  
}
```

```

        const synchronized =
            await readSynchronizationState();

        if (synchronized === true) {
            circuitBreaker.close();
        } else {
            circuitBreaker.open("UNSYNCHRONIZED");
        }

    } catch (error) {

        circuitBreaker.open("SYNC_UNKNOWN");
        recordVerificationFailure(error);

    }

    await sleep(60000);
}

```

The production implementation SHALL use a scheduler or timer mechanism appropriate to the selected runtime rather than relying on a blocking process.

7.4 Request Timeout

Each JSON-RPC request SHALL have a bounded timeout.

The adapter SHALL NOT permit a stalled RPC connection to indefinitely hold the translation pipeline open.

Example:

RPC timeout:
5 seconds

Polling interval:
60 seconds

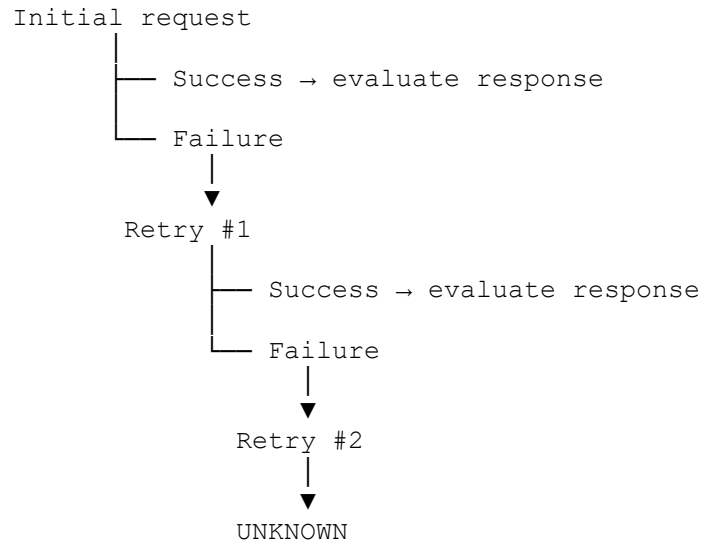
Verification result:
TRUE / FALSE / UNKNOWN

If the RPC endpoint fails to respond within the configured timeout, the result SHALL be UNKNOWN.

7.5 RPC Retry Policy

RPC retries SHALL be bounded.

Example:

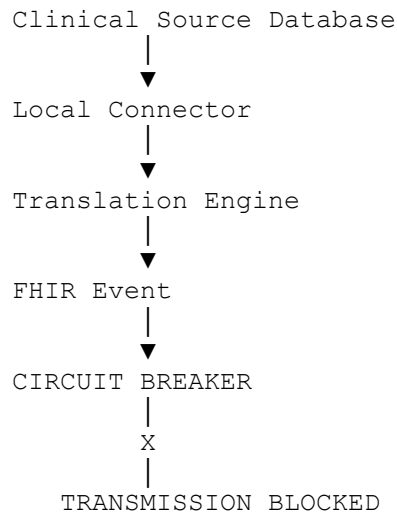


The adapter SHALL NOT continuously retry until an affirmative response is obtained.

Repeated retries must not become a mechanism for delaying circuit-breaker activation.

7.6 Fail-Closed Enforcement

When the circuit breaker enters OPEN state:



The adapter SHALL prevent:

- outbound Haven event submission;
- protocol synchronization-dependent database operations;
- cross-network translation;
- event forwarding to downstream Haven endpoints.

Local source-system operation SHALL remain under the client's control.

The circuit breaker SHALL therefore block the **Haven interoperability path**, not indiscriminately disable the client's entire EMR or database.

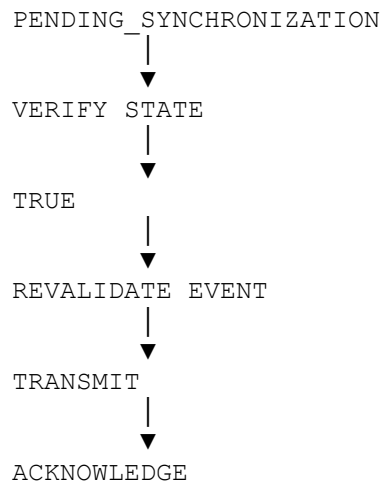
7.7 Event Handling During OPEN State

Events generated while the circuit is OPEN SHALL enter a bounded local pending queue.

Example state:

PENDING_SYNCHRONIZATION

When synchronization is restored:



Events SHALL NOT be transmitted merely because the circuit transitioned from OPEN to CLOSED.

Each event SHALL undergo the normal validation and cryptographic integrity checks before release.

7.8 Anti-Bypass Requirement

The following mechanisms SHALL NOT be accepted as synchronization evidence:

```
Client JavaScript variable
HTTP request parameter
Checkout success callback
Frontend Boolean
Client database flag
Application environment variable
Manually supplied API parameter
Local configuration override
Cached TRUE state beyond permitted TTL
```

Only independently obtained protocol state from the designated Polygon contract SHALL authorize the interoperability gate.

7.9 Cache Restriction

A previously verified TRUE result SHALL NOT be cached indefinitely.

The maximum synchronization validity window SHALL be bounded by the protocol configuration.

Under normal operation:

```
T0          → TRUE verified
T0+60s      → verification required
T0+120s     → verification required
T0+180s     → verification required
```

If the adapter cannot obtain a fresh valid result within the permitted validity window, it SHALL transition to OPEN.

7.10 Operational Principle

The circuit breaker is therefore not a payment UI control.

It is an **infrastructure authorization gate** positioned at the final edge between locally translated healthcare events and the Haven cross-network transport layer.

Its governing rule is:

```
NO VALID ON-CHAIN SYNCHRONIZATION
=
NO HAVEN CROSS-NETWORK TRANSMISSION
```

CRYPTOGRAPHIC SECURITY ARCHITECTURE

8.1 Cryptographic Boundary

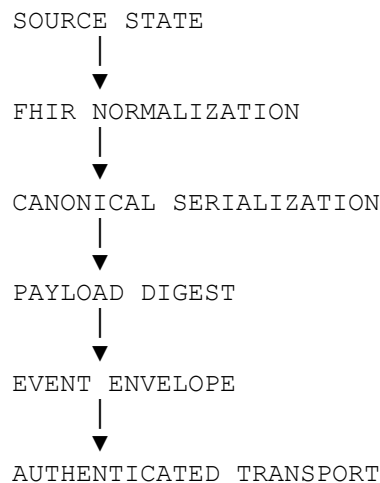
The STA SHALL establish a cryptographic boundary between:

1. the originating healthcare system;
2. the local translation environment;
3. the Haven transport layer; and
4. the public blockchain synchronization layer.

The Polygon network SHALL NOT receive raw clinical payloads.

Clinical information SHALL remain within the authorized healthcare interoperability boundary unless a separate, explicitly approved data-sharing mechanism requires otherwise.

The cryptographic architecture SHALL provide:



8.2 Cryptographic Objectives

The STA cryptographic subsystem SHALL provide:

- payload integrity;
- event authenticity;
- deterministic event identification;
- replay resistance;
- provenance binding;
- source-to-event traceability;
- protocol-version binding;
- tamper detection.

Cryptography SHALL not be represented as proof that a clinical assertion is medically correct.

It establishes integrity and origin characteristics for the data that the source system supplied.

8.3 Hashing

The adapter SHALL use an approved cryptographic hash function for event integrity.

SHA-256 SHALL be supported as the baseline protocol digest algorithm.

Example:

```
payload_digest =  
SHA256(  
    canonical_serialized_payload  
)
```

The resulting digest SHALL be represented using a canonical encoding.

Example:

```
sha256:<hexadecimal-digest>
```

The digest SHALL be calculated only after canonical serialization.

Hashing a non-canonical representation SHALL be prohibited for protocol integrity purposes.

8.4 Domain Separation

Protocol digests SHALL include sufficient domain information to prevent the same byte sequence from being ambiguously interpreted across protocol contexts.

Conceptual construction:

```
digest_input =  
"HAVEN-STAS-V1" ||  
event_type ||  
source_system_id ||  
mapping_version ||  
canonical_payload
```

The exact serialization grammar SHALL be version controlled.

Changing the serialization grammar SHALL result in a protocol-version transition.

8.5 Event Identification

Each translated event SHALL receive a globally unique event identifier.

Preferred format:

UUIDv4

Where deterministic identifiers are required for idempotency, a deterministic identifier MAY be derived from a defined source-system event key.

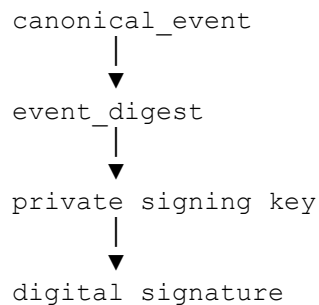
The event identifier SHALL never be derived solely from mutable clinical content where subsequent correction of that content could create ambiguous event identity.

8.6 Payload Signing

Where event-level signatures are required, the STA SHALL use an approved asymmetric signature scheme.

The signing key SHALL belong to the authorized adapter or participating enterprise identity.

Conceptually:



The corresponding verification key SHALL be distributed through the protocol's approved trust mechanism.

Private signing keys SHALL never be transmitted to:

- client applications;
- browser code;
- source databases;
- checkout interfaces;
- Haven API request bodies;
- Polygon smart contracts.

8.7 Key Storage

Private keys SHALL be stored using a secure key-management mechanism appropriate to the deployment environment.

Acceptable mechanisms MAY include:

Hardware Security Module
Trusted Platform Module
Operating-system secure keystore
Cloud HSM / KMS
Protected secret-management service

Plaintext private keys SHALL NOT be stored in:

Source code
Git repositories
Environment files committed to source control
Database tables
Application logs
FHIR payloads
Client-side JavaScript

Key rotation SHALL be supported without invalidating previously accepted events.

8.8 Transport Security

All external adapter communication SHALL use encrypted transport.

Minimum requirement:

TLS 1.2+

TLS 1.3 SHOULD be preferred where supported.

For Haven-controlled protocol endpoints, mutual TLS SHOULD be used where operationally feasible.

The STA SHALL validate:

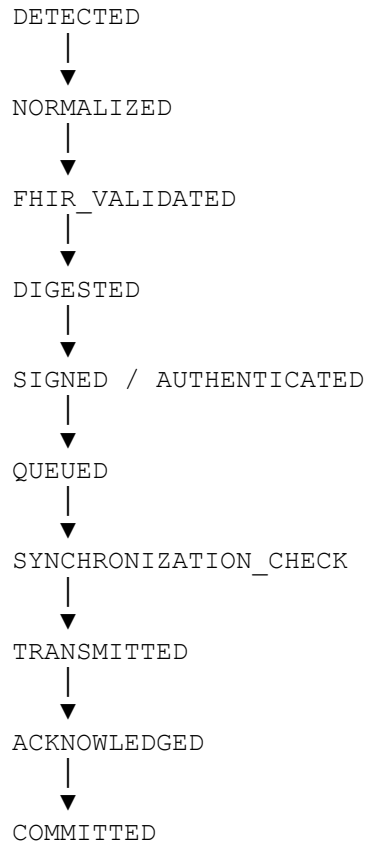
- certificate chain;
- certificate validity;
- hostname;
- protocol version;
- negotiated cipher policy.

Certificate validation failures SHALL prevent transmission.

EVENT INTEGRITY, IDEMPOTENCY AND REPLAY PROTECTION

9.1 Event Lifecycle

Every protocol event SHALL progress through a controlled lifecycle.



An event SHALL NOT be marked COMMITTED solely because an HTTP request was successfully sent.

A protocol acknowledgement SHALL be required.

9.2 Idempotency

The adapter SHALL prevent duplicate protocol events resulting from:

- network retries;
- process restarts;
- database cursor replay;
- duplicate webhook delivery;
- RPC delays;

- timeout ambiguity.

Every outbound event SHALL include an idempotency key.

Example:

```
Idempotency-Key:  
<protocol_event_id>
```

The receiving Haven endpoint SHALL treat repeated submissions bearing the same valid idempotency identity as the same logical event.

9.3 Transactional Processing Boundary

The adapter SHALL maintain a clear boundary between:

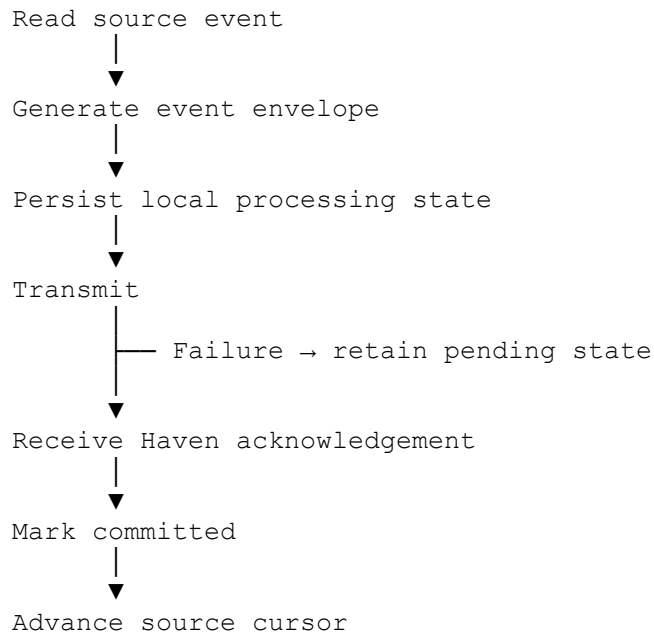
SOURCE ACKNOWLEDGEMENT

and

HAVEN COMMIT ACKNOWLEDGEMENT

The source event SHALL not be acknowledged as fully processed until the adapter has safely persisted the minimum state required to recover the event.

Preferred sequence:



9.4 Replay Protection

The protocol SHALL detect repeated submission of an already committed event.

Replay detection MAY use:

```
event_id
+
source_system_id
+
protocol_version
```

The exact replay identity tuple SHALL be defined by the Haven protocol core.

Events outside their permitted temporal or protocol validity window SHALL be rejected or quarantined according to protocol policy.

9.5 Timestamp Validation

Every event SHALL contain a source event timestamp and adapter processing timestamp where available.

The adapter SHALL validate timestamp format and detect impossible or materially inconsistent values.

Example:

```
sourceEventTimestamp
adapterReceivedTimestamp
adapterProcessedTimestamp
```

The source timestamp SHALL describe the originating event.

The adapter timestamp SHALL describe when the edge infrastructure processed it.

The two values SHALL never be substituted for one another.

9.6 Clock Synchronization

The STA host SHALL maintain synchronized system time using an approved NTP source.

Clock drift SHALL be monitored.

Material clock deviation SHALL generate an operational warning and MAY prevent transmission where protocol timestamp guarantees cannot be maintained.

The adapter SHALL NOT fabricate timestamps to compensate silently for incorrect client system clocks.

9.7 Provenance

Every translated event SHALL preserve sufficient provenance to reconstruct its origin.

Minimum provenance:

```
{
  "sourceSystemId": "CLIENT_EMR",
  "sourceRecordId": "opaque-id",
  "sourceConnector": "postgresql",
  "mappingVersion": "1.0.0",
  "adapterVersion": "1.0.0",
  "processedAt": "2026-08-30T00:00:00Z"
}
```

Provenance SHALL be minimized to the information required for interoperability, auditability, and technical traceability.

9.8 Tamper Detection

The adapter SHALL detect unauthorized modification of locally persisted protocol state where integrity protection is required.

Critical state MAY be protected using:

- Authenticated encryption
- HMAC
- Digital signatures
- Protected database storage
- Filesystem integrity controls

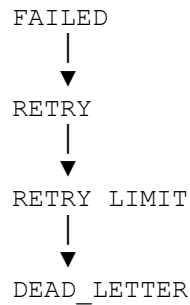
A detected integrity violation SHALL result in quarantine of the affected event or state record.

The adapter SHALL NOT automatically repair cryptographically invalid state using an unverified replacement.

9.9 Dead-Letter Handling

Events that repeatedly fail translation or transmission SHALL enter a controlled dead-letter state.

Example:



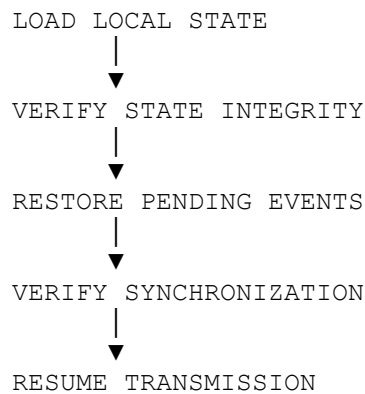
Dead-letter events SHALL retain:

- event identifier;
- failure category;
- failure timestamp;
- retry count;
- source reference;
- payload digest;
- mapping version;
- adapter version.

Clinical payload exposure within operational logs SHALL be minimized.

9.10 Recovery

Following adapter restart:



The adapter SHALL assume that any transmission whose final acknowledgement was not durably recorded may require safe replay.

Idempotency controls SHALL make such recovery deterministic.

9.11 Security Invariant

The complete event pipeline SHALL enforce:

```
VALID SOURCE EVENT
    AND
VALID FHIR REPRESENTATION
    AND
VALID CRYPTOGRAPHIC INTEGRITY
    AND
VALID TRANSPORT AUTHENTICATION
    AND
VALID ON-CHAIN SYNCHRONIZATION
    AND
VALID PROTOCOL ACKNOWLEDGEMENT
    =
COMMITTED HAVEN EVENT
```

Failure of any mandatory condition SHALL prevent the event from reaching COMMITTED state.

PRODUCTION DEPLOYMENT SPECIFICATION

10.1 Runtime Isolation

The STA SHALL operate as an independently managed service.

The preferred production deployment model is a hardened Linux host running the adapter inside a containerized runtime or an equivalently isolated system-service environment.

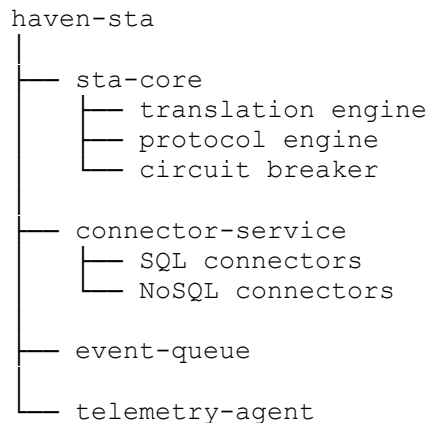
The deployment SHALL separate:

- Adapter Runtime
- Connector Runtime
- Local Queue
- Configuration
- Secrets
- Logs
- Monitoring

A single compromised connector process SHALL NOT automatically obtain unrestricted access to the adapter's cryptographic material.

10.2 Container Architecture

A reference container deployment MAY use:



Where operationally practical, connector and core processes SHOULD be isolated into separate containers.

Inter-container communication SHALL occur through authenticated local interfaces.

10.3 Immutable Application Image

Production STA application images SHOULD be immutable.

Runtime modification of executable application files SHALL be prohibited.

Deployment artifacts SHALL be versioned using:

```
Application Version
Container Image Digest
Protocol Version
Mapping Version
Configuration Version
```

Example:

```
STA:           1.0.0
Protocol:      1.0
Mapping:       1.0.0
Image Digest:  sha256:<digest>
Config Version: 1.0.0
```

10.4 Configuration Separation

Configuration SHALL be separated into three classes.

CLASS A — Infrastructure Configuration

Contains:

```
Database host
Database port
Local network interfaces
Resource limits
Logging destination
Monitoring destination
```

CLASS B — Protocol Configuration

Contains:

```
Polygon chain identifier
Synchronization contract address
Contract interface definition
Haven protocol endpoint
Protocol version
Cryptographic algorithm identifiers
Synchronization policy
```

CLASS C — Secrets

Contains:

Database credentials
API credentials
mTLS private keys
Signing keys
HMAC secrets
Certificate private material

Class C material SHALL never be embedded in the container image.

10.5 Environment Variable Restrictions

Environment variables MAY be used to inject non-sensitive deployment configuration.

Protocol-critical values SHOULD preferably be protected through a signed configuration artifact or equivalent integrity-controlled mechanism.

The STA SHALL distinguish between:

CONFIGURATION VALUE

and

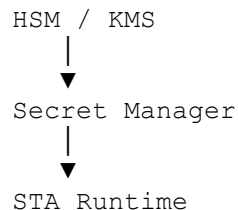
AUTHORIZATION STATE

A local configuration variable SHALL never be accepted as evidence that the Polygon synchronization state is TRUE.

10.6 Secret Management

Secrets SHALL be injected at runtime through an approved secret-management mechanism.

Preferred hierarchy:



Secrets SHALL not be:

- written to source control;
- printed in logs;
- included in exception traces;
- transmitted through query parameters;
- stored inside FHIR resources.

10.7 Filesystem Permissions

The STA process SHALL run under a dedicated non-root operating-system identity.

Required principle:

```
STA USER
├── READ → application binaries
├── READ/WRITE → operational queue
├── READ → approved configuration
├── READ → approved secret mount
└── NO ACCESS → unrelated system resources
```

The container or host filesystem SHOULD use read-only application volumes wherever possible.

10.8 Outbound Firewall Policy

The adapter SHALL use an allow-list-based outbound network policy.

Required destinations SHALL be limited to:

```
Approved Haven endpoints
Approved Polygon RPC endpoints
Approved certificate / identity infrastructure
Approved telemetry endpoint
Approved NTP service
```

All other outbound destinations SHOULD be denied by default.

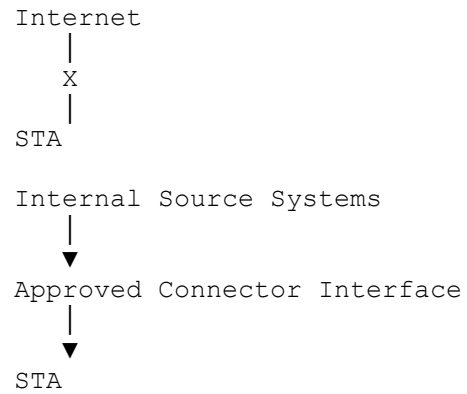
The adapter SHALL NOT require unrestricted internet access.

10.9 Inbound Firewall Policy

Inbound access to the STA SHALL be denied by default.

Only explicitly required management or local integration ports may be exposed.

Example logical policy:



Administrative access SHALL occur through the client's controlled management network.

SERVICE HEALTH, OBSERVABILITY AND TELEMETRY

11.1 Health Model

The STA SHALL expose internally observable health states without exposing sensitive clinical information.

Health SHALL be separated into:

```
PROCESS HEALTH
CONNECTOR HEALTH
PROTOCOL HEALTH
SYNCHRONIZATION HEALTH
QUEUE HEALTH
CRYPTOGRAPHIC HEALTH
```

A process may therefore be operational while protocol transmission is intentionally blocked.

11.2 Liveness Endpoint

The liveness check SHALL determine whether the STA process is functioning.

Conceptual response:

```
{
  "status": "alive"
}
```

A successful liveness result SHALL NOT imply that Haven transmission is authorized.

11.3 Readiness Endpoint

Readiness SHALL represent whether the adapter can perform its intended protocol function.

Example:

```
{
  "status": "ready",
  "translation": true,
  "synchronization": true,
  "rpc": true
}
```

If synchronization is FALSE:

```
{
  "status": "not_ready",
  "translation": false,
  "synchronization": false
}
```

The readiness endpoint SHALL NOT expose raw clinical records or cryptographic secrets.

11.4 Connector Health

Each connector SHALL expose an independent health state.

Example:

```
{
  "connector": "CLIENT_EMR",
  "engine": "postgresql",
  "status": "healthy",
  "latencyMs": 18
}
```

Database credentials and connection strings SHALL never be returned by health endpoints.

11.5 Synchronization Telemetry

The adapter SHALL record synchronization verification events.

Minimum telemetry:

```
verification_timestamp
chain_id
contract_address
participant_identifier
verification_result
rpc_latency
rpc_endpoint_identifier
failure_code
circuit_state
```

The participant identifier SHALL be represented in a privacy-preserving form where possible.

11.6 Circuit-Breaker Telemetry

Every circuit transition SHALL produce an operational event.

Example:

```
{
  "event": "CIRCUIT_STATE_CHANGE",
  "previous": "CLOSED",
  "current": "OPEN",
  "reason": "UNSYNCHRONIZED",
  "timestamp": "2026-08-30T00:00:00Z"
}
```

Permitted failure reasons SHALL be enumerated.

Example:

```
CHAIN_MISMATCH
CONTRACT_INVALID
RPC_TIMEOUT
RPC_ERROR
INVALID_RESPONSE
ABI_DECODE_FAILURE
UNSYNCHRONIZED
CONFIGURATION_INTEGRITY_FAILURE
CLOCK_DRIFT
```

11.7 Event Metrics

The adapter SHOULD expose aggregate metrics including:

```
events_detected_total
events_translated_total
events_rejected_total
events_transmitted_total
events_committed_total
events_failed_total
events_dead_lettered_total

translation_latency_ms
rpc_latency_ms
haven_latency_ms

queue_depth
retry_count
circuit_open_duration
synchronization_success_rate
```

Metrics SHALL be aggregate operational measurements.

Raw clinical payloads SHALL not be included.

11.8 Logging Policy

Logs SHALL use structured machine-readable formats.

Each log event SHOULD contain:

```
timestamp
severity
component
event_code
event_id
source_system_id
protocol_version
adapter_version
```

Clinical payload contents SHALL NOT be logged by default.

Sensitive identifiers SHALL be redacted, tokenized, or pseudonymized according to the deployment's data-minimization policy.

11.9 Audit Events

The adapter SHALL generate immutable or integrity-protected audit records for security-critical operations.

At minimum:

```
Adapter startup
Configuration load
Configuration verification
Database connector activation
FHIR mapping activation
Cryptographic key activation
Synchronization verification
Circuit transition
Event transmission
Event acknowledgement
Credential rotation
Software upgrade
Administrative configuration change
```

Audit records SHALL be timestamped using synchronized system time.

11.10 Monitoring Alerts

Operational monitoring SHOULD trigger alerts for:

```
Repeated RPC failures
Unexpected chain ID
Unexpected contract address
Circuit OPEN beyond threshold
Queue growth
Dead-letter accumulation
```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Database connector failure
Certificate expiration
Clock drift
Cryptographic verification failure
Unauthorized configuration modification

Alert thresholds SHALL be configurable at the operational layer but SHALL NOT permit disabling mandatory protocol security controls.

11.11 Observability Boundary

Observability systems SHALL never become a secondary clinical data repository.

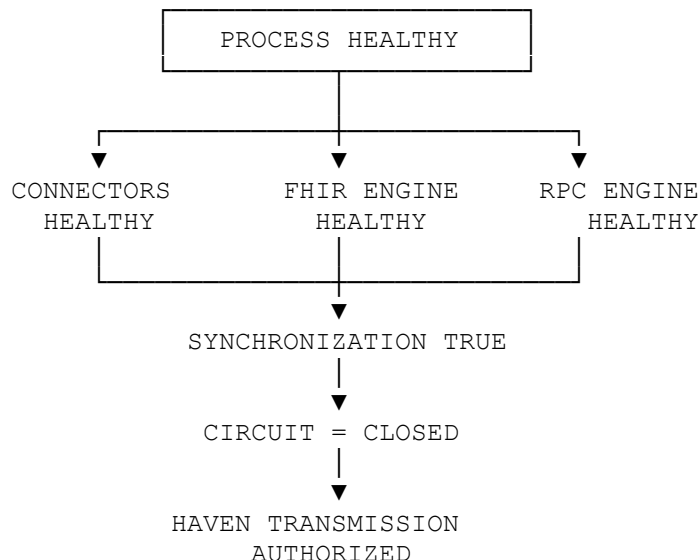
The telemetry layer SHALL answer:

Is the adapter functioning?
Is the source connector healthy?
Is translation succeeding?
Is synchronization valid?
Is the protocol reachable?
Are events accumulating?

It SHALL NOT require storage of complete clinical payloads to answer these questions.

11.12 Operational State Model

The complete STA health state can therefore be represented as:



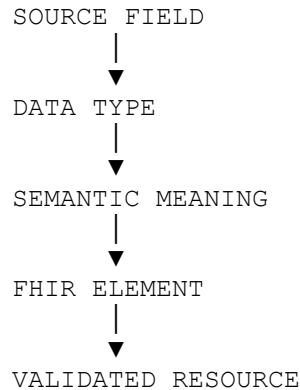
Any mandatory dependency entering an invalid state SHALL propagate a fail-closed condition to the outbound interoperability path.

CANONICAL DATA MAPPING SPECIFICATION

12.1 Mapping Philosophy

The STA SHALL translate source-system data into FHIR resources through deterministic mapping rules.

The mapping engine SHALL distinguish between:



A field SHALL NOT be mapped solely because its name resembles a FHIR element.

Semantic compatibility SHALL be established through the mapping registry.

12.2 Core Mapping Matrix

Source Domain	Typical Source Field	FHIR Resource	FHIR Element
Patient	patient_id	Patient	identifier
Patient	first_name	Patient	name.given
Patient	last_name	Patient	name.family
Patient	date_of_birth	Patient	birthDate
Patient	sex	Patient	gender
Patient	phone	Patient	telecom
Patient	email	Patient	telecom
Encounter	encounter_id	Encounter	identifier
Encounter	encounter_type	Encounter	class/type
Encounter	admission_time	Encounter	period.start
Encounter	discharge_time	Encounter	period.end
Practitioner	clinician_id	Practitioner	identifier
Practitioner	clinician_name	Practitioner	name

Source Domain	Typical Source Field	FHIR Resource	FHIR Element
Organization	facility_id	Organization	identifier
Organization	facility_name	Organization	name
Diagnosis	diagnosis_code	Condition	code
Diagnosis	diagnosis_text	Condition	code.text
Observation	observation_code	Observation	code
Observation	observation_value	Observation	value[x]
Observation	observation_unit	Observation	valueQuantity. unit
Laboratory	report_id	DiagnosticReport	identifier
Laboratory	report_status	DiagnosticReport	status
Medication	medication_code	Medication	code
Prescription	medication_order	MedicationRequest	medication[x]
Procedure	procedure_code	Procedure	code
Coverage	policy_id	Coverage	identifier
Claim	claim_id	Claim	identifier

The exact mapping SHALL be determined by the source-system schema and approved mapping profile.

12.3 Identifier Rules

Identifiers SHALL preserve namespace context.

Canonical representation:

```

identifier.system
  +
identifier.value

```

The `system` value SHALL identify the authority responsible for issuing the identifier.

Example:

```

{
  "system": "urn:haven:source:CLIENT_EMR",
  "value": "opaque-source-identifier"
}

```

The adapter SHALL NOT assume that two identical identifier values from different source systems represent the same entity.

12.4 Date and Time Rules

Source dates SHALL be normalized according to their semantic precision.

Examples:

2026-08-30

↓

FHIR date

2026-08-30 14:30:00

↓

FHIR dateTime

2026-08-30 14:30:00 +01:00

↓

FHIR dateTime with timezone

The adapter SHALL NOT manufacture timezone information where the source system provides none.

If timezone interpretation is required, it SHALL be supplied through an explicitly configured facility-level timezone.

12.5 Code System Mapping

Source clinical codes SHALL be translated using an approved terminology registry.

Example:

SOURCE CODE

↓

CODE SYSTEM IDENTIFICATION

↓

TERM LOOKUP

├

Exact Match → FHIR Coding

├

Approved Equivalent → FHIR Coding

└

No Valid Match → Unresolved

The adapter SHALL not silently substitute an approximate clinical code where semantic equivalence has not been established.

12.6 Quantity Mapping

Numeric observations SHALL preserve:

```
value
unit
system
code
```

Where the source system supplies a recognized unit, the adapter SHALL normalize it to the approved FHIR representation.

Conceptual output:

```
{
  "value": 120,
  "unit": "mmHg",
  "system": "http://unitsofmeasure.org",
  "code": "mm[Hg]"
}
```

12.7 Null Handling

Null, empty, unavailable, unknown, and not-applicable values SHALL remain semantically distinct where the FHIR resource model supports the distinction.

The adapter SHALL NOT convert all missing values into:

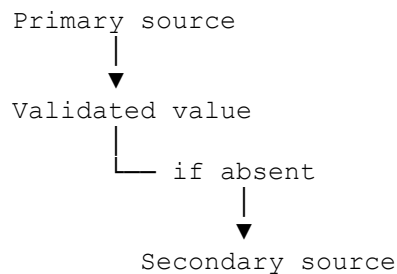
```
"unknown"
```

without establishing that this is semantically correct.

12.8 Source Precedence

Where multiple source fields contain the same conceptual value, the mapping profile SHALL define precedence.

Example:



The adapter SHALL not arbitrarily select one value based on field order.

CLINICAL EVENT TAXONOMY AND ERROR HANDLING

13.1 Event Classification

The STA SHALL classify incoming source changes into protocol-defined event categories.

Minimum categories:

PATIENT_CREATED
PATIENT_UPDATED

ENCOUNTER_CREATED
ENCOUNTER_UPDATED
ENCOUNTER_CLOSED

OBSERVATION_CREATED
OBSERVATION_UPDATED

CONDITION_CREATED
CONDITION_UPDATED

PROCEDURE_CREATED
PROCEDURE_UPDATED

MEDICATION_CREATED
MEDICATION_UPDATED
MEDICATION_DISCONTINUED

DIAGNOSTIC_REPORT_CREATED
DIAGNOSTIC_REPORT_UPDATED

COVERAGE_CREATED
COVERAGE_UPDATED

CLAIM_CREATED
CLAIM_UPDATED

The taxonomy SHALL be extensible without changing the fundamental adapter architecture.

13.2 Event Classification Rules

Event classification SHALL be determined from authoritative source-system state transitions.

The adapter SHALL distinguish:

CREATE
UPDATE
DELETE / RETRACTION
CORRECTION
VOID
MERGE

A correction SHALL not automatically be interpreted as a new unrelated clinical event.

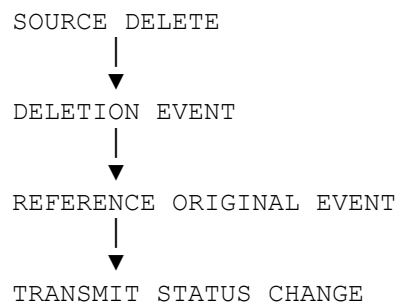
Where FHIR semantics require a replacement or revised resource, the mapping profile SHALL define the relationship explicitly.

13.3 Deletion Handling

A source-system deletion SHALL NOT necessarily result in physical deletion of a previously transmitted Haven event.

Instead, the adapter SHALL create the appropriate protocol retraction, deletion, or status-change event according to the Haven event model.

Conceptually:



This preserves event traceability.

13.4 Validation Error Classes

Validation failures SHALL be categorized.

MAP_001	Missing required source field
MAP_002	Invalid source datatype
MAP_003	Unsupported source value
MAP_004	Terminology mapping failure
MAP_005	FHIR cardinality violation
MAP_006	Invalid identifier
MAP_007	Invalid date/time
MAP_008	Invalid reference
MAP_009	Unsupported resource type
MAP_010	Schema version mismatch

Cryptographic failures SHALL use a separate namespace:

CRYPTO_001	Digest generation failure
CRYPTO_002	Signature failure
CRYPTO_003	Signature verification failure
CRYPTO_004	Key unavailable
CRYPTO_005	Integrity verification failure

Synchronization failures SHALL remain separately identifiable:

SYNC_001	Chain mismatch
SYNC_002	Contract mismatch
SYNC_003	RPC timeout
SYNC_004	RPC error
SYNC_005	Invalid contract response
SYNC_006	Unsynchronized participant
SYNC_007	Synchronization state unknown

13.5 Error Severity

Errors SHALL be classified as:

FATAL
BLOCKING
RETRYABLE
NON_BLOCKING

FATAL

Requires operator or deployment intervention.

Example:

Invalid protocol configuration

BLOCKING

Prevents the affected event from transmission.

Example:

FHIR validation failure

RETRYABLE

Temporary infrastructure condition.

Example:

RPC timeout

NON_BLOCKING

Does not prevent processing of the event.

Example:

Optional telemetry endpoint unavailable

13.6 Malformed Source Data

Malformed source data SHALL never be silently transformed into valid-looking clinical data.

The adapter SHALL:

```
Detect
↓
Classify
↓
Quarantine
↓
Record reason
↓
Await correction / retry
```

The original source record SHALL remain untouched unless the adapter has an explicitly authorized write-back operation.

13.7 Ambiguous Mapping

When multiple FHIR interpretations are possible and no mapping rule establishes semantic precedence, the event SHALL be marked:

MAPPING_AMBIGUOUS

The adapter SHALL not select an arbitrary interpretation.

13.8 Partial Resource Failure

If a composite event contains multiple independently translatable resources:

```
Encounter
├── Patient
├── Practitioner
├── Condition
└── Observation
```

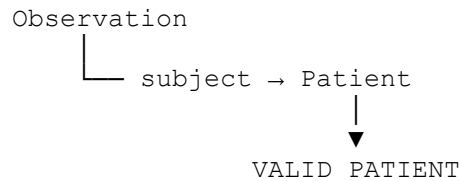
failure of one resource SHALL not automatically corrupt successfully validated resources.

The event coordinator SHALL preserve dependency relationships and determine whether the complete event requires atomic transmission.

13.9 Dependency Validation

FHIR references SHALL be validated before transmission.

Example:

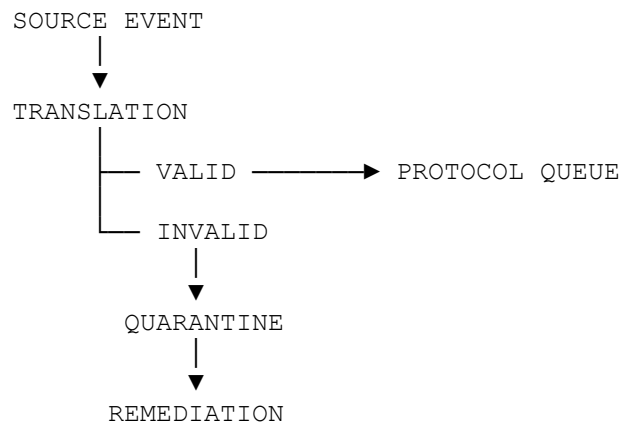


An unresolved mandatory reference SHALL block the affected event.

13.10 Quarantine Queue

Rejected events SHALL enter a controlled quarantine queue.

Example:



Quarantine records SHALL include the failure classification and event digest while minimizing duplication of sensitive clinical data.

13.11 Recovery After Correction

Once the underlying source data or mapping configuration has been corrected, the event SHALL be reprocessed through the complete validation pipeline.

It SHALL NOT bypass:

FHIR validation
Cryptographic packaging
Synchronization verification
Idempotency controls

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

Protocol acknowledgement

A previously rejected event SHALL therefore enter the same security path as a newly detected event.

13.12 Clinical Integrity Rule

The governing translation principle SHALL be:

UNKNOWN ≠ TRUE

AMBIGUOUS ≠ VALID

MALFORMED ≠ NORMALIZED

UNVERIFIED ≠ AUTHORIZED

The STA exists to preserve interoperability integrity, not to manufacture certainty where the originating system does not provide it.

HAVEN PROTOCOL TRANSMISSION INTERFACE

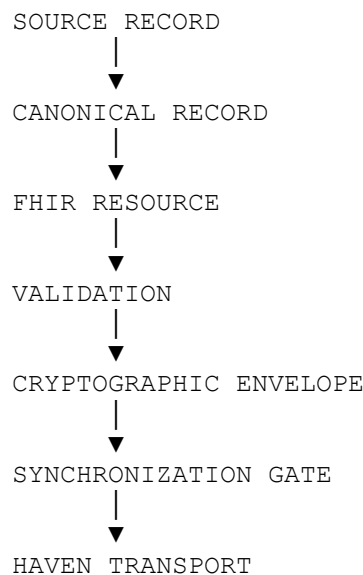
14.1 Interface Boundary

The STA SHALL expose a controlled outbound interface to the Haven coordination layer.

The transmission interface SHALL accept only validated protocol event envelopes.

The interface SHALL NOT accept raw source-database records as protocol events.

Mandatory processing boundary:



No component downstream of the synchronization gate may re-enable an event rejected by the gate.

14.2 Transport Protocol

The preferred transport shall be:

HTTPS
+
TLS 1.2 or higher
+
mTLS where supported
+
JSON protocol envelope

The Haven endpoint SHALL authenticate the participating adapter before accepting protocol events.

14.3 Endpoint Model

The canonical interface MAY expose an endpoint equivalent to:

```
POST /protocol/v1/events
```

The exact production URL SHALL be supplied through protected protocol configuration.

The adapter SHALL NOT dynamically accept arbitrary destination URLs from client applications.

14.4 Request Headers

Minimum protocol headers:

```
Content-Type: application/json
Accept: application/json
X-Haven-Protocol-Version: 1
X-Haven-Adapter-Version: <version>
X-Haven-Event-Id: <event-id>
Idempotency-Key: <event-id>
```

Authentication credentials SHALL be transmitted only through the approved authentication mechanism.

14.5 Request Envelope

Example:

```
{
  "protocolVersion": "1.0",
  "eventId": "7d4f3b7e-....",
  "sourceSystemId": "CLIENT_EMR",
  "eventType": "Observation.created",
  "eventTimestamp": "2026-08-30T00:00:00Z",
  "mappingVersion": "1.0.0",
  "resourceType": "Observation",
  "payloadDigest": "sha256:<digest>",
  "payload": {},
  "provenance": {
    "adapterVersion": "1.0.0"
  }
}
```

The receiver SHALL independently validate the event rather than trusting the adapter's local validation result.

14.6 Authentication

Authentication SHALL establish the identity of the participating adapter.

Supported mechanisms MAY include:

```
mTLS certificate
Signed request
Short-lived access token
Workload identity
```

Long-lived static bearer credentials SHOULD be avoided where stronger workload identity mechanisms are available.

14.7 Authentication Failure

Authentication failure SHALL prevent event transmission.

Example:

```
401 / 403
  |
  ▼
AUTHENTICATION_FAILURE
  |
  ▼
EVENT RETAINED
  |
  ▼
NO COMMIT
```

The adapter SHALL not repeatedly retry an invalid credential indefinitely.

Credential failures SHALL generate an operational alert.

14.8 Protocol Response

A successful Haven response SHALL contain an explicit acknowledgement.

Example:

```
{
  "status": "accepted",
  "eventId": "7d4f3b7e-....",
  "ackId": "ack-....",
  "protocolVersion": "1.0"
}
```

The adapter SHALL verify that the returned event identifier corresponds to the submitted event.

A successful HTTP status without a valid protocol acknowledgement SHALL NOT be treated as a committed event.

14.9 Response Classes

The adapter SHALL classify responses into:

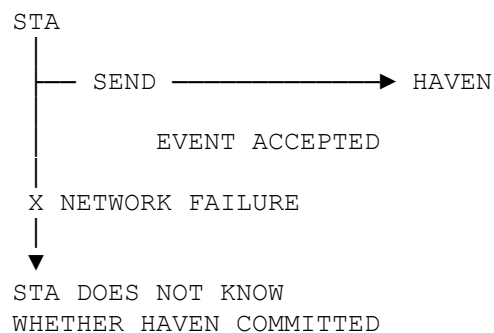
2xx → Protocol response evaluation
4xx → Permanent / policy failure
5xx → Retryable server failure
Timeout → Unknown delivery state
Network failure → Unknown delivery state

A 4xx response SHALL not automatically be retried indefinitely.

A 5xx response MAY be retried using bounded exponential backoff.

14.10 Unknown Delivery State

The most important transmission ambiguity is:



The adapter SHALL NOT automatically generate a second event with a new event identifier.

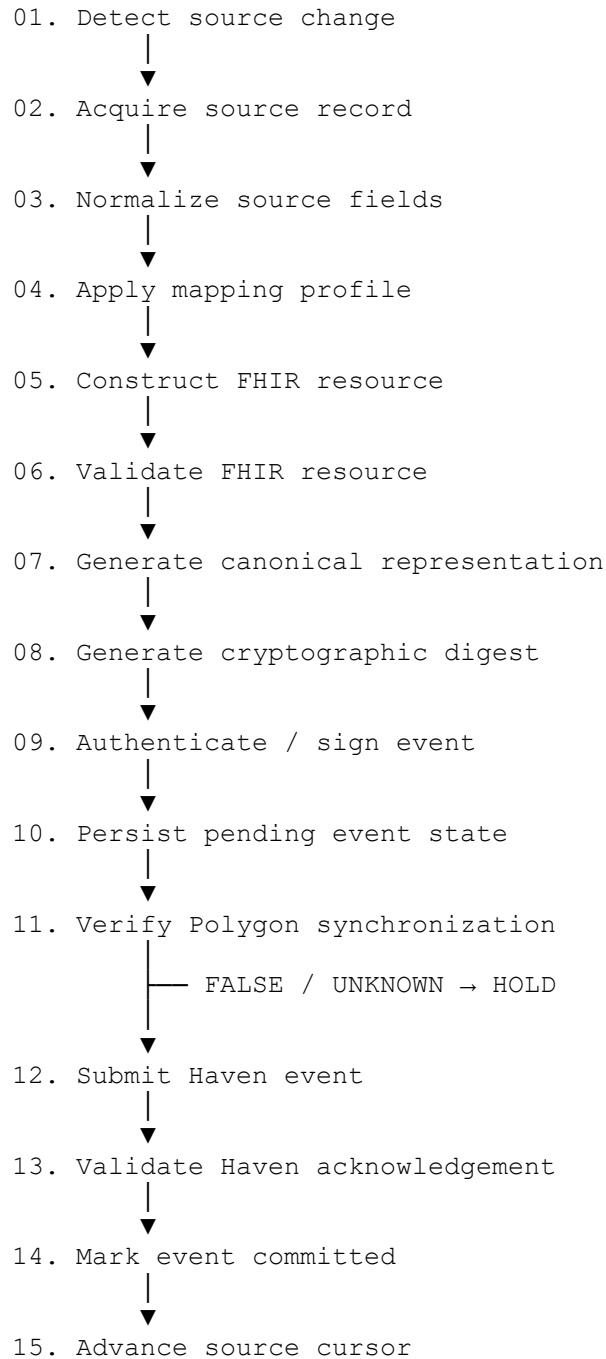
It SHALL retry using the original idempotency identity.

The Haven receiving layer SHALL determine whether the event was already accepted.

END-TO-END EVENT TRANSMISSION CONTROL

15.1 Complete Processing Sequence

The production STA SHALL execute the following sequence:



The ordering SHALL be preserved.

15.2 Pre-Transmission Gate

Before transmission, the adapter SHALL evaluate:

```
sourceEventValid
AND
fhirValid
AND
digestValid
AND
authenticationValid
AND
chainVerified
AND
contractVerified
AND
rpcVerified
AND
synchronizationState == TRUE
AND
eventNotCommitted
```

Only when every mandatory predicate evaluates TRUE may transmission proceed.

15.3 Authorization Decision

Conceptual implementation:

```
function mayTransmit(event) {

    return (
        event.sourceValid === true &&
        event.fhirValid === true &&
        event.digestValid === true &&
        event.authenticated === true &&
        sync.chainVerified === true &&
        sync.contractVerified === true &&
        sync.rpcVerified === true &&
        sync.state === true &&
        event.committed !== true
    );
}
```

The return value SHALL be generated internally by the STA.

No client request parameter may force:

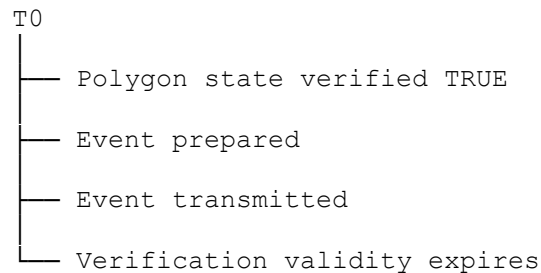
```
mayTransmit = true
```

15.4 Synchronization Race Condition

Because the Polygon state may change after a successful poll but before transmission, the adapter SHALL recognize a synchronization verification interval rather than treating the result as permanent authorization.

The adapter SHALL therefore maintain a bounded synchronization validity window.

Example:



If the configured validity window expires before transmission, the adapter SHALL perform a fresh synchronization check.

15.5 Immediate Circuit Opening

The adapter SHALL immediately transition to OPEN when it receives an explicit negative synchronization result.

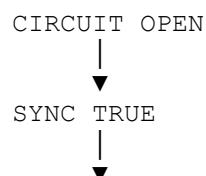
It SHALL also transition to OPEN when mandatory verification becomes indeterminate.

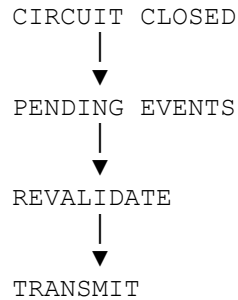
Examples:

FALSE
RPC_TIMEOUT
RPC_ERROR
INVALID_ABI_RESPONSE
CHAIN_MISMATCH
CONTRACT_MISMATCH
CONFIGURATION_INTEGRITY_FAILURE

15.6 Queued Event Release

When synchronization returns TRUE:





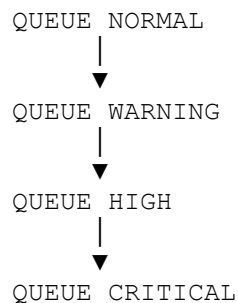
Queued events SHALL be released according to configured ordering and priority rules.

Events SHALL retain their original event timestamps.

15.7 Backpressure

The adapter SHALL protect itself against queue exhaustion.

When queue utilization approaches the configured threshold:



At CRITICAL state, the adapter SHALL stop accepting additional protocol work into local persistence unless sufficient capacity can be safely established.

The adapter SHALL never silently discard queued clinical events to recover capacity.

15.8 Ordering

Where event ordering is clinically or operationally significant, the adapter SHALL maintain source-system ordering metadata.

Example:

```
source_sequence = 1001  
source_sequence = 1002  
source_sequence = 1003
```

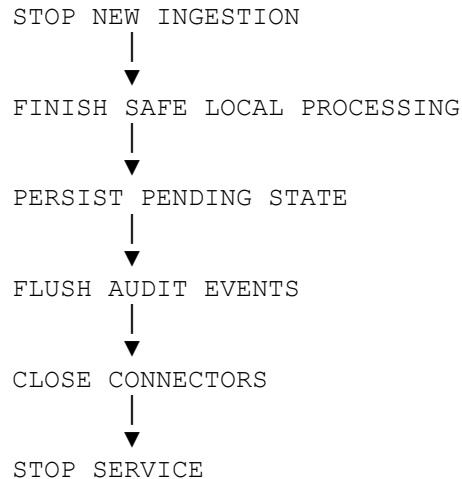
Events SHALL not be arbitrarily reordered across a dependency chain.

Independent events MAY be transmitted concurrently where the protocol permits parallel processing.

15.9 Graceful Shutdown

The adapter SHALL support graceful shutdown.

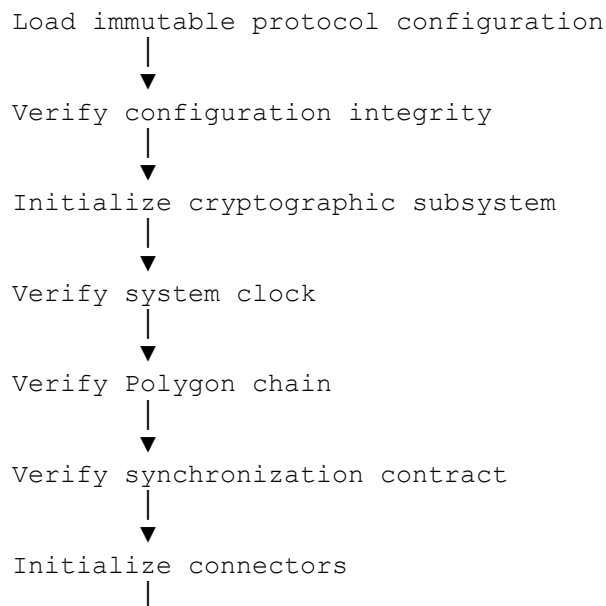
Shutdown sequence:

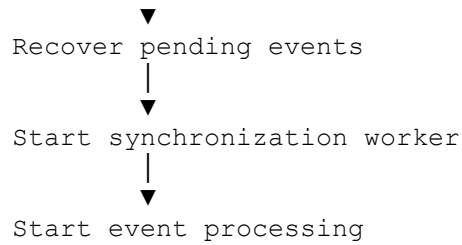


The adapter SHALL never interpret process shutdown as successful event commitment.

15.10 Startup Sequence

Startup SHALL execute:

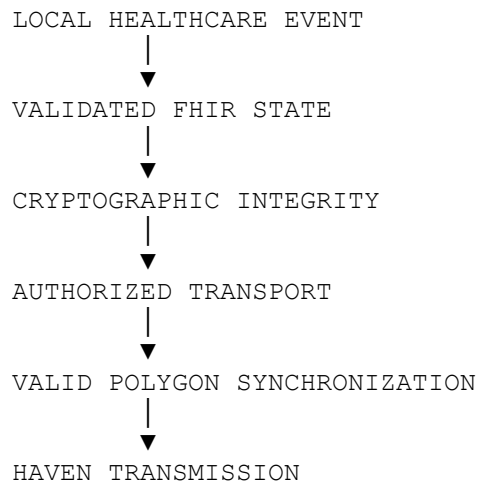




Until the mandatory initialization checks complete, the adapter SHALL remain in a non-transmitting state.

15.11 Final Enforcement Rule

The STA SHALL enforce the following protocol invariant at the edge:



No single client-side application component SHALL possess sufficient authority to remove the synchronization gate while retaining access to the Haven interoperability path.

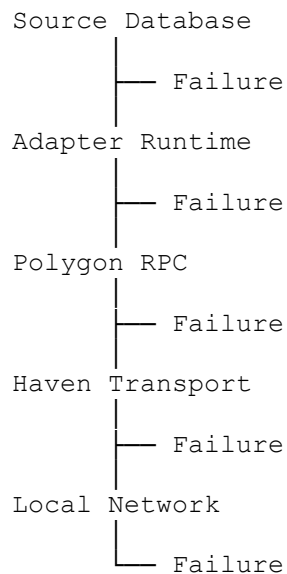
The adapter is therefore the final technical enforcement boundary between a participating healthcare facility's local systems and the Haven coordination network.

DISASTER RECOVERY AND RESILIENCE

16.1 Failure-Domain Separation

The STA SHALL assume that any individual dependency may fail independently.

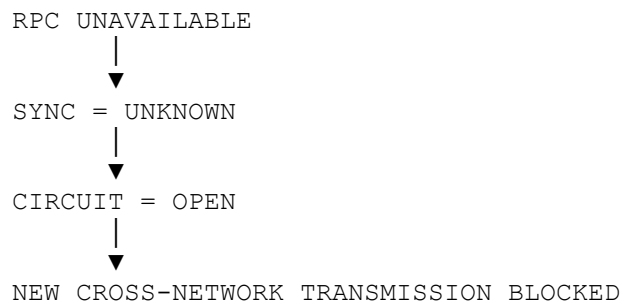
The following failure domains SHALL be treated separately:



Failure of one dependency SHALL NOT corrupt state belonging to another dependency.

16.2 RPC Failure

If the Polygon JSON-RPC service becomes unavailable:



Existing locally persisted events SHALL remain intact.

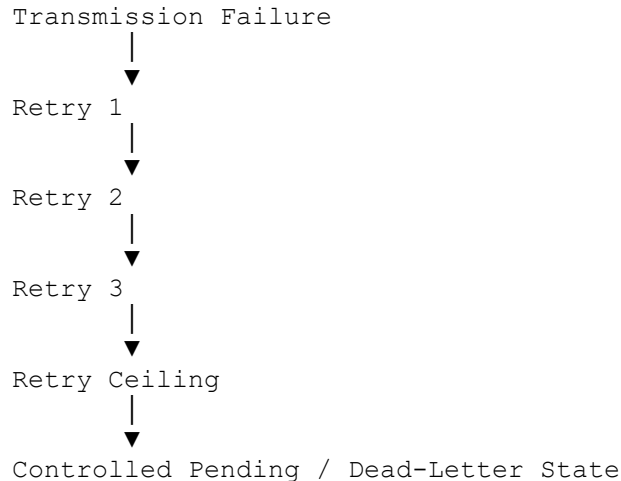
The adapter SHALL periodically retry synchronization verification according to the configured bounded retry policy.

16.3 Haven Endpoint Failure

If the Haven transport endpoint becomes unavailable, validated events SHALL remain in the local pending queue.

The adapter SHALL use bounded exponential backoff.

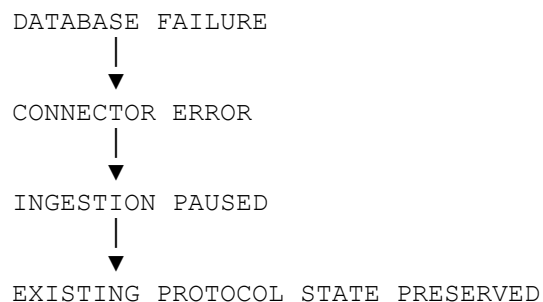
Example:



Repeated failures SHALL not create duplicate events.

16.4 Local Database Failure

If the source database becomes unavailable:



The adapter SHALL not fabricate source records to maintain event flow.

When database connectivity is restored, incremental extraction SHALL resume from the last safely committed source cursor.

16.5 Adapter Restart Recovery

The adapter SHALL persist sufficient operational state to recover after:

- process termination;
- host restart;
- container replacement;
- operating-system failure;
- controlled deployment;
- unexpected power loss.

Startup recovery SHALL identify:

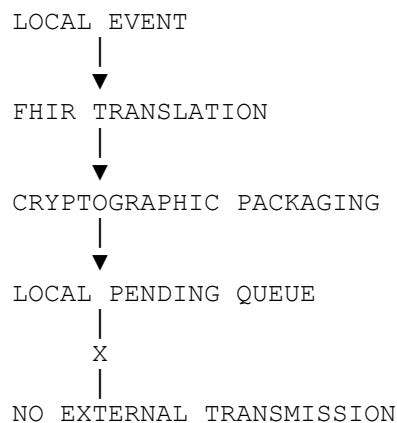
Pending events
Unacknowledged events
Last source cursor
Last synchronization state
Dead-letter events
Incomplete processing operations

Events with uncertain delivery status SHALL be retried using their original idempotency identity.

16.6 Offline Facility Operation

Temporary loss of external connectivity SHALL not require modification of source-system records.

During offline operation:



Once connectivity is restored, the event SHALL undergo a fresh synchronization check before transmission.

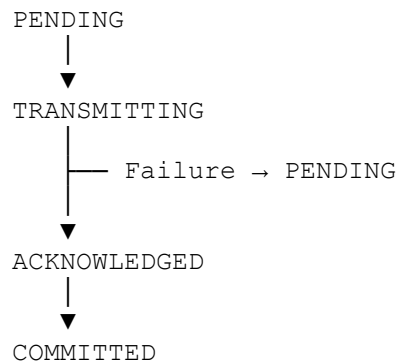
16.7 Queue Durability

The pending queue SHALL use durable storage where event recovery is required.

Queue records SHALL be protected against partial writes.

The implementation SHOULD use transactional persistence or equivalent atomic state transitions.

Example:

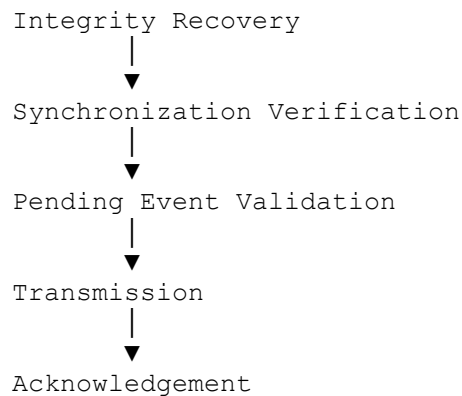


An event SHALL never transition directly from PENDING to COMMITTED without a valid acknowledgement.

16.8 Recovery Ordering

Recovery SHALL prioritize protocol correctness over throughput.

Preferred order:



The adapter SHALL not flush an entire backlog blindly immediately after reconnection.

16.9 Backup Requirements

Operational state required for recovery SHALL be backed up according to the participating organization's infrastructure policy.

Backups SHALL include, where applicable:

- Protocol configuration
- Mapping configuration
- Operational queue metadata
- Audit records
- Cursor state
- Certificate metadata
- Deployment manifests

Private cryptographic keys SHALL be backed up only through the approved key-management mechanism.

Keys SHALL not be copied into ordinary backup archives.

16.10 Recovery Point Objective

The deployment SHOULD establish an explicit Recovery Point Objective (RPO) for protocol state.

Critical event state SHOULD be persisted before the adapter reports successful local processing.

16.11 Recovery Time Objective

The deployment SHOULD establish an operational Recovery Time Objective (RTO).

The target SHALL be determined by:

- Event volume
- Clinical workflow dependency
- Network availability
- Database recovery time
- Infrastructure redundancy
- Queue depth

RTO SHALL not be achieved by disabling mandatory synchronization controls.

SECURITY HARDENING AND ADVERSARIAL RESISTANCE

17.1 Threat Model

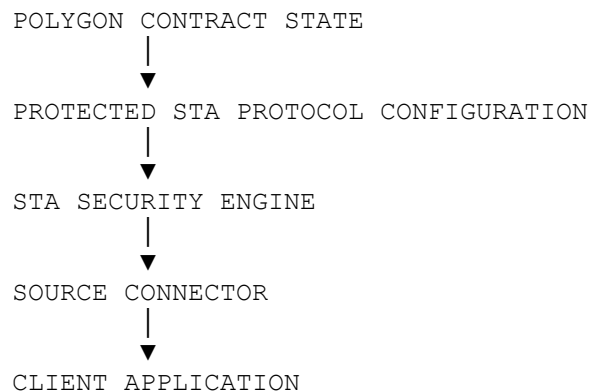
The STA SHALL assume that a participating client environment may contain:

- compromised application code;
- malicious developers;
- unauthorized administrators;
- modified checkout logic;
- manipulated configuration;
- compromised credentials;
- rogue database processes;
- intercepted network traffic;
- replayed events;
- malicious or malformed payloads.

The architecture SHALL therefore minimize the trust granted to the client application layer.

17.2 Trust Hierarchy

The protocol SHALL establish the following conceptual trust hierarchy:



The client application SHALL occupy a lower trust position than the protocol enforcement layer.

17.3 Client-Side Payment Bypass

A client SHALL NOT be able to authorize Haven transmission by modifying a checkout implementation.

The following shall have no authorization authority:

```
checkoutFee = 0
feeEnabled = false
skipFee = true
paymentComplete = true
isNetworkSynchronized = true
```

These values may exist inside a client application, but they SHALL be irrelevant to the STA's independently obtained synchronization state.

17.4 Adapter Tampering

The production deployment SHALL implement application integrity controls.

Recommended controls:

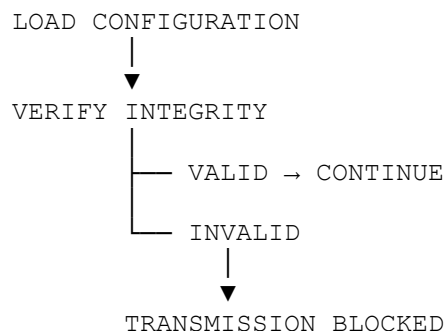
```
Signed deployment artifacts
Container image digest pinning
Read-only application filesystem
Least-privilege runtime
Host integrity monitoring
Configuration integrity verification
Restricted administrative access
Centralized audit logging
```

A detected modification of protocol-critical binaries SHALL cause the adapter to enter a non-transmitting state.

17.5 Configuration Tampering

Protocol configuration SHALL be integrity protected.

At startup and during defined runtime checks:



The adapter SHALL not silently fall back to an unverified configuration.

17.6 Contract Substitution Attack

An attacker SHALL NOT be able to redirect synchronization verification to an arbitrary contract.

The adapter SHALL verify:

```
Expected Chain ID
+
Pinned Contract Address
+
Expected Function Interface
+
Valid ABI Response
```

A mismatch SHALL produce:

```
SYNC_CONTRACT_VERIFICATION_FAILURE
```

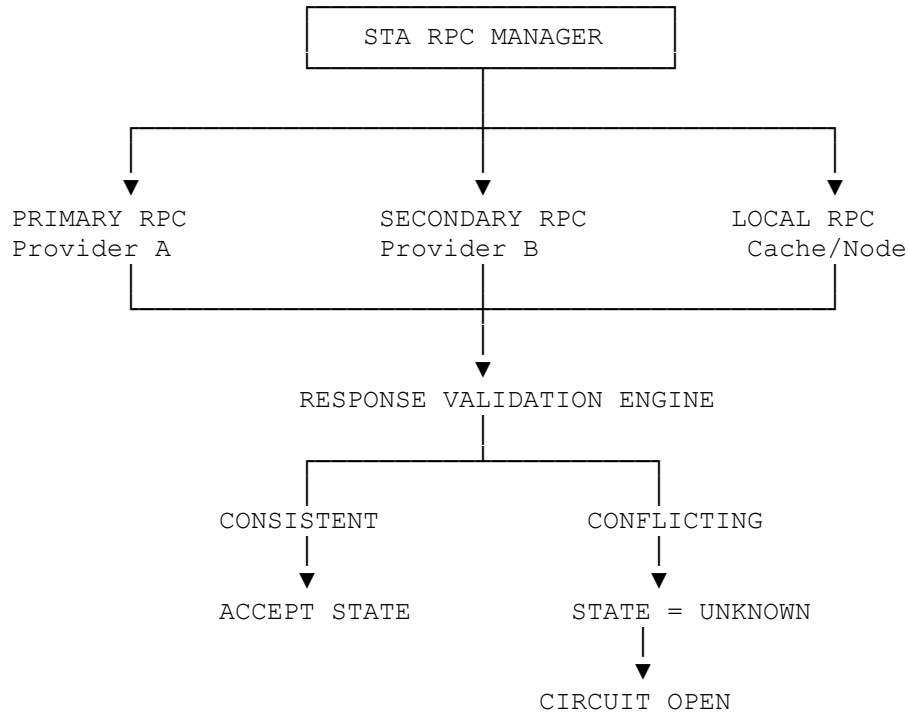
and SHALL open the circuit.

17.7 RPC Resilience and Multi-Endpoint Verification

The STA SHALL NOT depend upon a single Polygon JSON-RPC endpoint for synchronization verification.

The production deployment SHALL maintain a controlled RPC endpoint cluster consisting of multiple independently reachable providers.

Reference topology:



The RPC manager SHALL support deterministic failover.

If the primary endpoint becomes unavailable, times out, or returns an invalid response, the STA SHALL attempt the next approved endpoint according to the configured failover policy.

Example endpoint configuration:

```
RPC_ENDPOINTS = [
  {
    "name": "primary",
    "url": "https://polygon-mainnet.g.alchemy.com/v2/<KEY>",
    "priority": 1
  },
  {
    "name": "secondary",
    "url": "https://polygon-rpc.com",
    "priority": 2
  },
  {
    "name": "local",
    "url": "http://127.0.0.1:8545",
    "priority": 3
  }
]
```

The actual production implementation MAY use the provider/failover abstraction supported by the deployed `web3.py` version or an equivalent RPC-management implementation.

17.7.1 Endpoint Eligibility

An RPC endpoint SHALL NOT become an eligible synchronization source merely because it responds to HTTP requests.

Before accepting an endpoint as authoritative for synchronization verification, the STA SHALL validate:

```
Endpoint Reachability
      +
RPC Response Validity
      +
Expected Polygon Chain ID
      +
Expected Contract Address
      +
Expected Contract Interface
      +
Valid isNetworkSynchronized Response
```

Failure of any mandatory validation SHALL invalidate that endpoint for the current verification attempt.

17.7.2 Chain Identity Verification

Every RPC endpoint SHALL independently return the expected Polygon chain identifier.

Conceptually:

```
chain_id = w3.eth.chain_id

if chain_id != EXPECTED_POLYGON_CHAIN_ID:
    raise SynchronizationVerificationError(
        "CHAIN_ID_MISMATCH"
    )
```

The STA SHALL NOT infer network identity from the hostname or endpoint name.

A correctly responding RPC endpoint connected to the wrong EVM network SHALL be treated as invalid.

17.7.3 Contract Identity Verification

After establishing chain identity, the STA SHALL verify that the configured synchronization contract corresponds to the pinned production contract address.

The adapter SHALL NOT accept a contract address supplied dynamically by the client application.

Conceptual validation:

```
if configured_contract_address != PINNED_CONTRACT_ADDRESS:
    raise SynchronizationVerificationError(
        "CONTRACT_ADDRESS_MISMATCH"
    )
```

The configured contract identity SHALL be protected by the STA's configuration-integrity mechanism.

17.7.4 Synchronization State Verification

The STA SHALL query the Polygon contract directly for the synchronization state associated with the participating network identity.

Conceptually:

```
sync_state = contract.functions.isNetworkSynchronized(
    participant_address
).call()
```

The returned value SHALL be interpreted as:

```
TRUE
    → synchronization condition satisfied

FALSE
    → synchronization condition not satisfied

INVALID / ERROR / UNKNOWN
    → synchronization cannot be established
```

The adapter SHALL never convert an RPC error into a TRUE synchronization result.

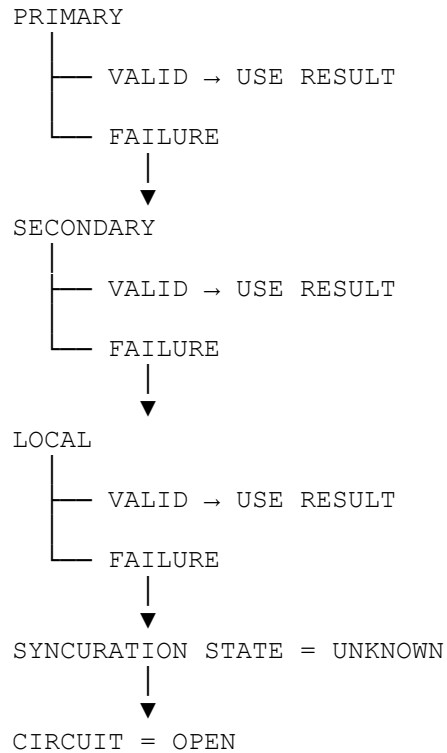
17.7.5 Endpoint Failure and Failover

A provider SHALL be considered failed for the current verification cycle when it produces:

```
Connection Failure
Timeout
Malformed JSON-RPC Response
RPC Error
Unexpected Chain ID
Invalid Contract Response
ABI Decode Failure
Contract Identity Failure
```

The RPC manager SHALL then attempt the next eligible endpoint.

Example:



17.7.6 Conflicting Provider Responses

Where multiple independent RPC providers return contradictory synchronization states, the STA SHALL NOT select the response that permits transmission.

Example:

Provider A → TRUE
Provider B → FALSE

The synchronization state SHALL be classified as:

CONFLICTING

and SHALL resolve to:

SYNCHRONIZATION = UNKNOWN
TRANSMISSION = BLOCKED
CIRCUIT = OPEN

A reconciliation mechanism MAY subsequently determine the canonical state.

17.7.7 Local RPC / Read-Cache Restrictions

A local Polygon node, RPC proxy, or read-cache MAY be included as an additional endpoint.

However, local availability SHALL NOT itself establish trust.

The STA SHALL perform the same:

```
Chain-ID Verification
Contract Verification
Response Validation
Synchronization-State Validation
```

against the local endpoint.

A local endpoint SHALL therefore function as an additional data source, not as an independent authorization authority.

17.7.8 Provider Independence

Where practical, primary and secondary RPC providers SHOULD operate across independent infrastructure domains.

The deployment SHOULD avoid constructing a nominally redundant cluster in which every endpoint ultimately depends upon the same underlying infrastructure provider.

The objective is:

```
Provider A Failure
    ≠
Provider B Failure
    ≠
Local Infrastructure Failure
```

17.7.9 RPC Health Monitoring

The STA SHALL maintain endpoint health metadata:

```
endpoint_id
priority
last_success_timestamp
last_failure_timestamp
consecutive_failures
average_latency
```

```
chain_id
contract_verification_status
current_health_state
```

Health metadata SHALL be operational telemetry and SHALL NOT be interpreted as synchronization authorization.

17.7.10 Failover Does Not Bypass the Circuit Breaker

RPC failover exists to improve availability.

It SHALL NOT weaken the fail-closed synchronization policy.

Therefore:

```
RPC FAILOVER
    ≠
SECURITY BYPASS
```

If no approved endpoint can establish a valid synchronization state, the STA SHALL remain OPEN.

17.7.11 Security Invariant

The final RPC authorization rule SHALL be:

```
VALID RPC RESPONSE
    +
EXPECTED POLYGON CHAIN
    +
EXPECTED CONTRACT
    +
VALID SYNCHRONIZATION RESPONSE
    =
ELIGIBLE SYNCHRONIZATION EVIDENCE
```

If any condition is absent:

```
SYNCHRONIZATION = UNKNOWN
```

and:

```
CROSS-NETWORK TRANSMISSION = BLOCKED
```

17.8 Replay Attack

The receiving protocol SHALL reject duplicate event identities after successful commitment.

The adapter SHALL preserve the original event identifier during retries.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

A retry SHALL therefore represent:

SAME EVENT

rather than:

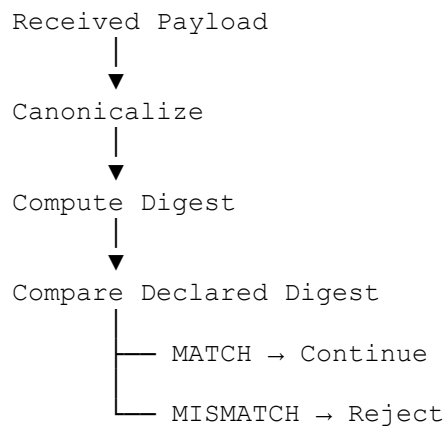
NEW EVENT

17.9 Payload Manipulation

Any modification between canonicalization and transmission SHALL invalidate the payload digest or signature.

The receiving side SHALL independently recompute the expected digest where applicable.

Conceptual verification:



17.10 Privilege Escalation

The STA SHALL follow least privilege.

Separate identities SHOULD be used for:

Database access
Protocol transmission
Cryptographic operations
Administrative operations
Monitoring

Compromise of one identity SHALL not automatically provide authority over all other functions.

17.11 Penetration Testing

Before production deployment, the STA SHALL undergo security assessment covering at minimum:

- API authentication
- Database connector security
- SQL injection resistance
- NoSQL injection resistance
- FHIR parser abuse
- Payload manipulation
- Replay attacks
- Credential exposure
- Configuration tampering
- Container escape
- Privilege escalation
- RPC manipulation
- Contract substitution
- Circuit-breaker bypass
- Queue poisoning
- Denial-of-service conditions

Testing SHALL include both authenticated and unauthenticated attack paths.

17.12 Circuit-Breaker Bypass Testing

The following scenarios SHALL be explicitly tested:

Polygon state = FALSE
Expected result → TRANSMISSION BLOCKED

Polygon RPC unavailable
Expected result → TRANSMISSION BLOCKED

Wrong chain ID
Expected result → TRANSMISSION BLOCKED

Wrong contract address
Expected result → TRANSMISSION BLOCKED

Malformed ABI response
Expected result → TRANSMISSION BLOCKED

Client checkout modified
Expected result → STA independently determines state

Client-side Boolean forged
Expected result → IGNORED

Local configuration altered
Expected result → INTEGRITY FAILURE / BLOCKED

17.13 Security Acceptance Criterion

The adapter SHALL pass the following architectural test:

IF A CLIENT DEVELOPER CAN REMOVE THE REQUIRED
PAYMENT ROUTING LOGIC FROM THE CLIENT APPLICATION

BUT THE STA STILL VERIFIES THE REQUIRED
ON-CHAIN SYNCHRONIZATION STATE INDEPENDENTLY,

THEN THE CLIENT APPLICATION MODIFICATION
MUST NOT AUTHORIZE HAVEN TRANSMISSION.

The enforcement boundary therefore resides outside the client application's declared payment variables.

17.14 Final Security Principle

The STA SHALL enforce:

CLIENT APPLICATION

≠

PROTOCOL AUTHORITY

LOCAL BOOLEAN

≠

ON-CHAIN STATE

HTTP SUCCESS

≠

PROTOCOL COMMIT

FHIR VALID

≠

TRANSMISSION AUTHORIZED

RPC ERROR

≠

SYNCHRONIZED

UNKNOWN

=

FAIL-CLOSED

The Stateless Translation Adapter is considered secure only when compromise or modification of the surrounding client application cannot independently manufacture authorization for Haven cross-network transmission.

PRODUCTION CONFORMANCE AND ACCEPTANCE

18.1 Conformance Objective

A production STA deployment SHALL be considered conformant only when the implementation satisfies the functional, security, interoperability, resilience, and synchronization requirements defined within this specification.

Conformance SHALL be demonstrated through documented technical testing rather than configuration declaration alone.

18.2 Deployment Acceptance Matrix

Domain	Acceptance Requirement	Required Result
Source Connectivity	Approved SQL/NoSQL connector establishes controlled access	PASS
Least Privilege	Connector cannot perform unauthorized administrative operations	PASS
Incremental Processing	Source events can be consumed without uncontrolled full-table polling	PASS
FHIR Translation	Supported records produce valid FHIR resources	PASS
Mapping Integrity	Mapping version is identifiable and integrity protected	PASS
Cryptography	Event digest is generated deterministically	PASS
Transport	TLS and approved authentication are enforced	PASS
Idempotency	Replayed events do not create duplicate commitments	PASS
Polygon Connectivity	Correct Polygon network is verified	PASS
Contract Verification	Expected synchronization contract is verified	PASS
Synchronization	<code>isNetworkSynchronized</code> state is independently obtained	PASS
Circuit Breaker	FALSE state blocks cross-network transmission	PASS
RPC Failure	RPC failure results in fail-closed behavior	PASS
Configuration Integrity	Protocol configuration tampering blocks transmission	PASS
Recovery	Pending events survive adapter restart	PASS
Observability	Security and operational state is auditable	PASS
Queue Safety	Events are not silently discarded	PASS
Upgrade Control	Protocol-critical changes require controlled deployment	PASS

18.3 Functional Test Suite

The deployment SHALL execute at minimum:

```
TEST-001  SQL source ingestion
TEST-002  NoSQL source ingestion
TEST-003  Incremental cursor recovery
TEST-004  FHIR Patient mapping
TEST-005  FHIR Encounter mapping
TEST-006  FHIR Observation mapping
TEST-007  Invalid FHIR resource rejection
TEST-008  Cryptographic digest verification
TEST-009  Duplicate event detection
TEST-010  Haven acknowledgement processing
TEST-011  Polygon chain verification
TEST-012  Synchronization TRUE
TEST-013  Synchronization FALSE
TEST-014  RPC timeout
TEST-015  RPC malformed response
TEST-016  Contract-address mismatch
TEST-017  Circuit-breaker transition
TEST-018  Pending-event recovery
TEST-019  Adapter restart
TEST-020  Configuration-integrity failure
```

Each test SHALL record:

```
Test ID
Environment
Software Version
Configuration Version
Input Condition
Expected Result
Observed Result
Timestamp
Operator / Automation Identity
PASS / FAIL
```

18.4 Synchronization Acceptance Test

The synchronization mechanism SHALL be tested independently of the client checkout implementation.

The test SHALL establish:

```
CLIENT CHECKOUT
    |
    X
    |
MODIFIED / BYPASSED
```

while the STA independently queries:

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

```

POLYGON CONTRACT
  |
  ▼
isNetworkSynchronized(participant)
  |
  ▼
FALSE

```

Expected result:

```

CIRCUIT = OPEN
TRANSMISSION = BLOCKED

```

The client checkout's local interpretation SHALL have no effect on the result.

18.5 Positive Synchronization Test

The inverse test SHALL establish:

```

POLYGON CONTRACT
  |
  ▼
VALID TRUE
  |
  ▼
STA VERIFICATION
  |
  ▼
CIRCUIT = CLOSED

```

A properly validated event SHALL then be permitted to enter the Haven transmission path.

18.6 Negative-Path Dominance

Security testing SHALL verify that negative conditions override positive application conditions.

Example:

```

Client Payment = SUCCESS
      +
Polygon Sync = FALSE
      =
TRANSMISSION BLOCKED

```

Likewise:

```

Client Payment = SUCCESS
      +
Polygon Sync = UNKNOWN
      =
TRANSMISSION BLOCKED

```

and:

```
Client Payment = FAILED
      +
Polygon Sync = TRUE
      =
EVENT MUST NOT BE FABRICATED
```

The adapter SHALL never create payment evidence merely because synchronization is available.

18.7 Performance Acceptance

Production validation SHALL measure:

```
Source extraction latency
Translation latency
FHIR validation latency
Cryptographic processing latency
RPC latency
Haven transmission latency
End-to-end event latency
Queue throughput
Recovery throughput
```

Performance optimization SHALL never weaken:

```
Authentication
Authorization
Integrity validation
Synchronization verification
Idempotency
Auditability
```

18.8 Capacity Testing

The STA SHALL be load-tested beyond expected production volume.

Testing SHALL establish:

```
Normal operating throughput
Peak event throughput
Queue saturation threshold
RPC saturation behavior
Database connection ceiling
Memory behavior
CPU behavior
Recovery behavior
```

The system SHALL fail predictably under excessive load.

Unbounded queue growth SHALL be treated as a production incident condition.

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

VERSIONING, UPGRADE CONTROL AND FINAL ARCHITECTURAL SPECIFICATION

19.1 Version Domains

The STA SHALL maintain independent versions for:

Adapter Version
Protocol Version
FHIR Version
Mapping Version
Configuration Version
Connector Version
Contract Interface Version

Example:

Adapter: 1.0.0
Protocol: 1.0
FHIR: R4
Mapping: 1.2.0
Configuration: 1.0.3
Contract ABI: 1.0

A change to one domain SHALL not be represented as an undocumented change to another.

19.2 Semantic Versioning

Software components SHOULD use semantic versioning:

MAJOR.MINOR.PATCH

Interpretation:

MAJOR
Breaking architectural or interface change

MINOR
Backward-compatible functionality

PATCH
Backward-compatible defect or security correction

Protocol-breaking changes SHALL require explicit compatibility analysis.

19.3 Mapping Version Control

A change to a FHIR mapping SHALL increment the mapping version.

Previously processed events SHALL retain the mapping version under which they were generated.

Example:

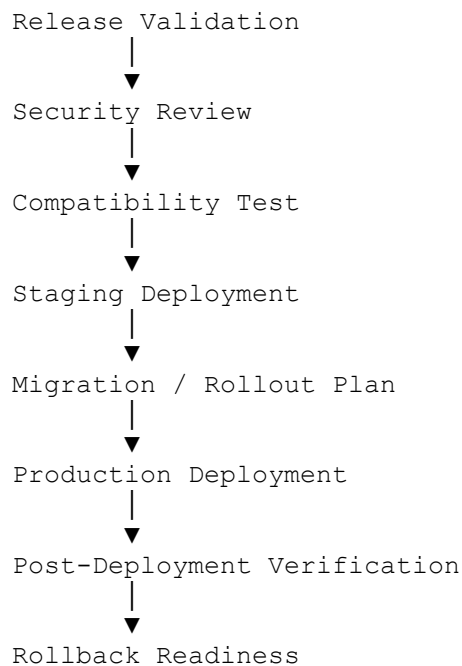
Event A → Mapping 1.0.0

Event B → Mapping 1.1.0

Historical event interpretation SHALL therefore remain reproducible.

19.4 Upgrade Procedure

Production upgrades SHALL follow:



19.5 Protocol-Critical Upgrade Restrictions

The following components SHALL receive heightened change control:

Synchronization logic
Contract address
Contract ABI
Circuit-breaker logic
Cryptographic verification
Protocol endpoints

Authentication mechanisms
Event serialization
FHIR mapping engine

No production operator SHALL modify these components through an ordinary client-facing checkout configuration.

19.6 Contract Address Transition

If the Haven protocol ever requires migration to a new synchronization contract, the adapter SHALL NOT silently replace the contract address.

A formal protocol migration SHALL define:

Old Contract
New Contract
Effective Block / Time
Protocol Version
Migration Procedure
Compatibility Window
Rollback Procedure
Verification Method

The migration SHALL be distributed through an authenticated and integrity-protected deployment mechanism.

19.7 Rollback

Every production upgrade SHALL have a defined rollback strategy.

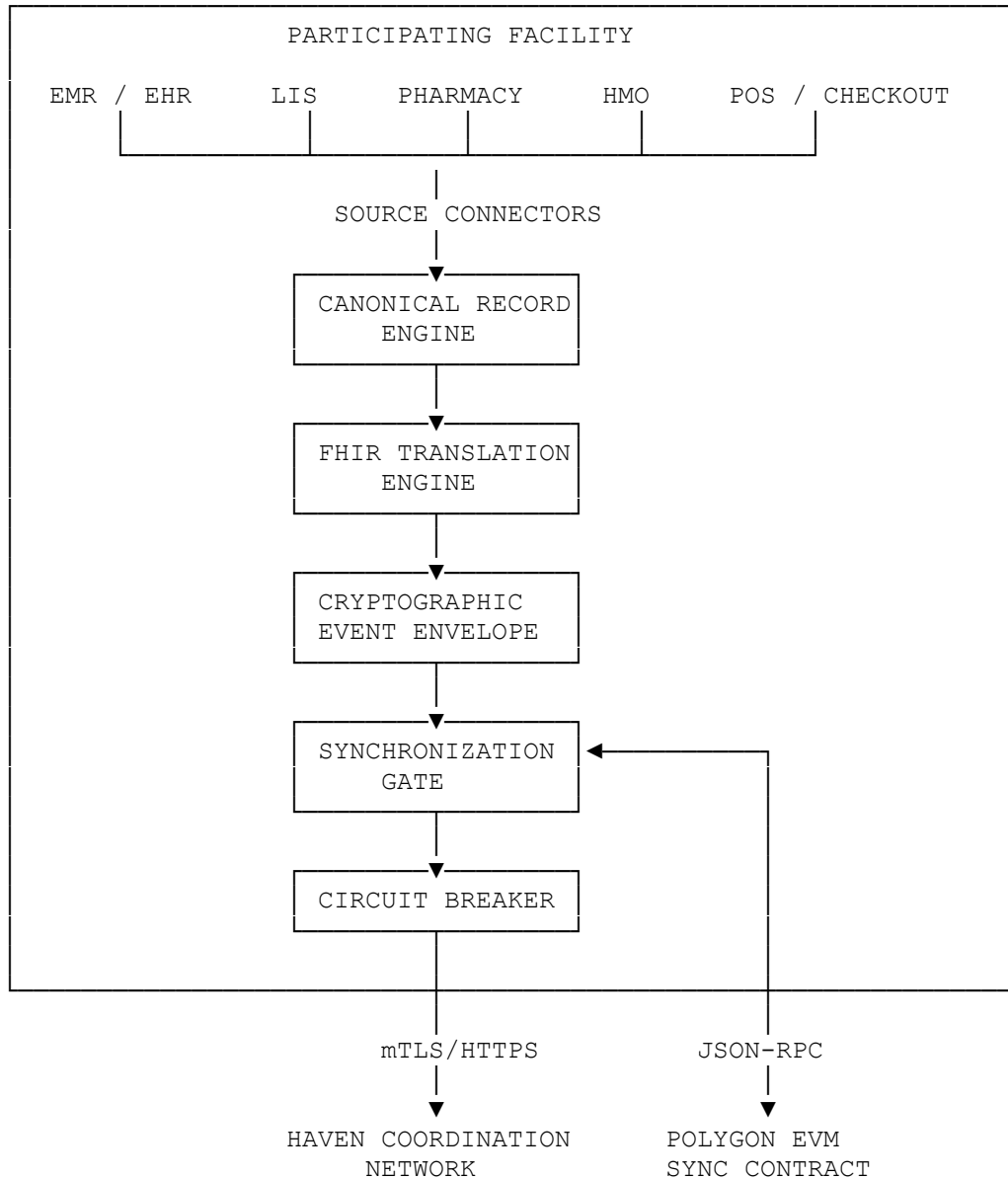
Rollback SHALL preserve:

Pending Events
Event IDs
Source Cursors
Audit Records
Cryptographic Integrity
Idempotency State

Rollback SHALL not permit an event previously committed to be represented as uncommitted without an explicit protocol reconciliation procedure.

19.8 Final Reference Architecture

The completed STAS architecture is:



19.9 Architectural Invariants

The following invariants **SHALL** govern every conformant implementation:

1. Client applications are not protocol authorities.

2. Raw clinical databases remain under client control.
3. The STA does not become the clinical system of record.
4. FHIR translation is deterministic and versioned.
5. Ambiguous data is not silently fabricated.
6. Clinical payloads are not written to the public blockchain merely to establish synchronization.
7. Synchronization state is independently obtained from Polygon.
8. Client-side payment variables cannot authorize transmission.
9. FALSE and UNKNOWN synchronization states fail closed.
10. RPC failure cannot be converted into authorization.
11. Duplicate transmission cannot create duplicate logical events.
12. Cryptographic integrity is verified before commitment.
13. Successful HTTP delivery does not equal protocol commitment.
14. Configuration tampering cannot silently alter protocol authority.
15. Recovery cannot bypass synchronization verification.
16. Mandatory security controls cannot be disabled through ordinary client configuration.

19.10 Final STAS Control Equation

The operational authorization condition SHALL be represented conceptually as:

```
HAVEN_TRANSMISSION_AUTHORIZED
=
SOURCE_EVENT_VALID
^
FHIR_RESOURCE_VALID
^
CRYPTOGRAPHIC_INTEGRITY_VALID
^
TRANSPORT_AUTHENTICATED
^
CHAIN_ID_VALID
^
CONTRACT_IDENTITY_VALID
^
RPC_RESPONSE_VALID
^
IS_NETWORK_SYNCHRONIZED == TRUE
```

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.

$\wedge$
EVENT_NOT_ALREADY_COMMITTED

If any mandatory predicate evaluates FALSE or UNKNOWN:

HAVEN_TRANSMISSION_AUTHORIZED = FALSE

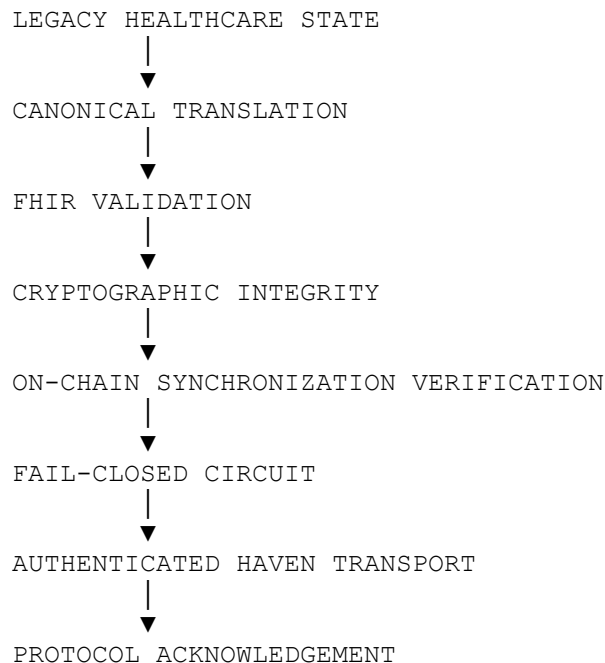
19.11 Final Specification Statement

The Haven Stateless Translation Adapter is defined as a **controlled edge interoperability execution layer** capable of translating heterogeneous healthcare-system state into standardized FHIR-native protocol events while maintaining cryptographic integrity, source provenance, asynchronous processing, and independently verified protocol synchronization.

Its defining security property is separation of **client application behavior from protocol authorization**.

The client environment may generate, modify, or process local healthcare data; however, the client application SHALL NOT possess unilateral authority to authorize cross-network Haven transmission.

The STA therefore establishes the final enforcement boundary through which every protocol event must pass:



END OF DOCUMENT 2

STATELESS TRANSLATION ADAPTER SPECIFICATIONS — STAS v2.0

Copyright © 2026 Dr. Eze Obinna (EO). All Rights Reserved. Haven Protocol Architecture Suite.
Licensed under GNU AGPL-3.0 rules.